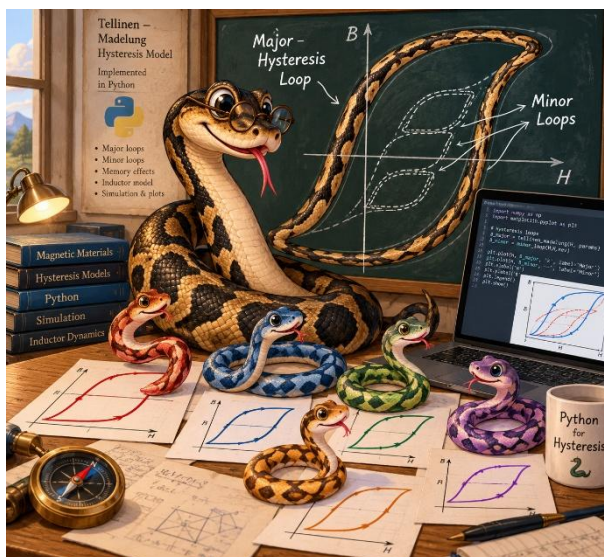


Модель магнитного гистерезиса Tellinen–Madelung и её реализация на Python



Оглавление

Введение	3
Исходная модель магнитного гистерезиса Tellinen	5
Включение Madelung магнитной памяти	7
Корневая траектория и структура магнитной памяти	7
Первый разворот и возвратная кривая первого порядка	8
Второй и последующие развороты	8
Привязка Tellinen-ветви к Madelung return point	8
Wiping-out и восстановление parent branch	9
Численное вычисление интегралов	10
Метод трапеций с конечными разностями	10
Более точный метод на основе квадратичной интерполяции Лагранжа	10
Алгоритмическая реализация модели Tellinen–Madelung	12
Структура Python-проекта Tellinen–Madelung	14
main.py — основной управляющий модуль	15
BH.py — ядро магнитной модели	17
Parameters.py	22
Parameters.txt	22

Hscan.py.....	23
Major_Loop.csv	23
Сравнение расчетов по моделям Tellinen–Madelung и Chan–Engelhardt.....	24
Модель индуктора с магнитным сердечником.....	26
Алгоритмическая реализация расчёта индуктора.....	29
Структура Python-проекта для расчёта индуктора	31
main.py — основной модуль расчёта индуктора	32
Поиск интервала для метода бисекции	32
Решение нелинейного уравнения	33
Фиксация подтверждённого состояния	34
Parameters.py и Parameters.txt	34
V.py.....	35
Выходные файлы	35
Сравнение расчетов с симуляциями индуктора в LTspice.....	35
Источники	43

Введение

Магнитный гистерезис принципиально отличается от обычной нелинейной зависимости $B(H)$ тем, что магнитная индукция определяется не только текущим значением напряжённости поля, но и предысторией его изменения. При одном и том же H материал может находиться в различных магнитных состояниях в зависимости от того, по какой траектории он пришёл в рассматриваемую точку. Поэтому корректная модель должна описывать не только форму мажорной петли, но и механизм формирования внутренних ветвей при последовательных разворотах поля. В [обзоре](#) Mörée и Leijon [1] подчёркивается, что именно способ хранения магнитной истории является одним из главных признаков, различающих современные модели гистерезиса.

В предлагаемой модели форма локальных ветвей строится на основе модели Tellinen. Её основная идея состоит в том, что дифференциальная проницаемость внутренней траектории определяется через наклон соответствующей ветви мажорной петли, умноженный на коэффициент, характеризующий положение текущей точки между верхней и нижней границами петли. При движении поля вверх используется производная нижней ветви мажорной петли, а при движении вниз — производная верхней. Благодаря этому внутренние траектории непосредственно связаны с геометрией реальной мажорной петли и естественным образом ограничиваются областью между её двумя ветвями. В отличие от некоторых более простых масштабируемых моделей, такое построение существенно уменьшает риск появления нефизических ветвей, выходящих за пределы мажорного цикла. Однако стандартная модель Tellinen обладает только локальной памятью: дальнейшее движение определяется текущими значениями H , B и направлением изменения поля. Этого достаточно для построения отдельных возвратных кривых, но недостаточно для строгого воспроизведения сложной последовательности вложенных минорных циклов. После нескольких разворотов модель сама по себе не хранит информацию о том, к какой из более ранних точек возврата должна прийти текущая ветвь.

Для устранения этого ограничения в предлагаемой модели используется принцип магнитной памяти Madelung, продемонстрированной на Рис. 1. История намагничивания представляется как последовательность точек разворота $P_1, P_2, \dots, P_n$, в которых знак изменения магнитного поля меняется на противоположный. Новая ветвь, возникающая из последней точки P_n , должна возвращаться к предыдущей точке P_{n-1} . Если этот возврат состоялся и движение продолжается дальше, соответствующий внутренний цикл удаляется из памяти, а траектория восстанавливает более раннюю ветвь. Эти свойства известны как «память точки возврата» (return-point memory) и «стирание памяти» (wiping-out memory). В модели Madelung именно последовательность точек разворота является компактным представлением магнитной предыстории.

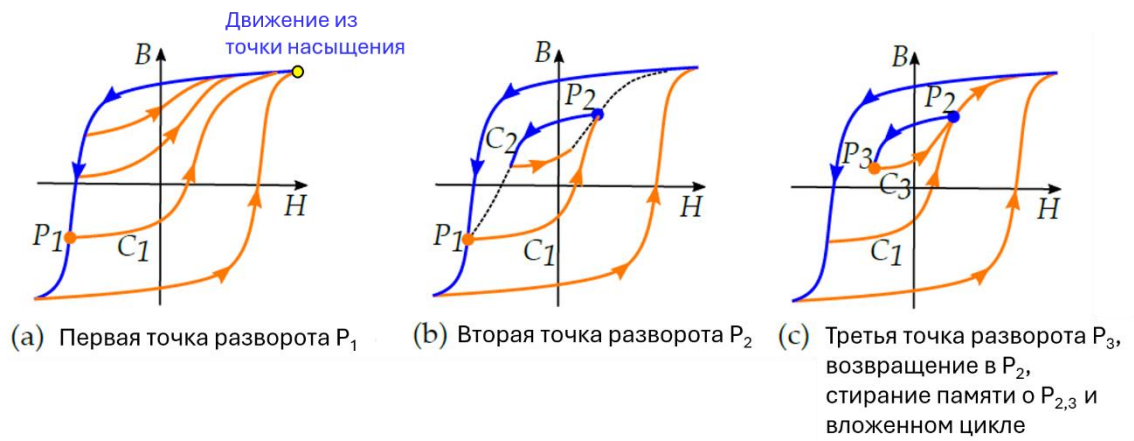


Рис. 1. Правила Madelung магнитной памяти.

Таким образом, две части модели выполняют разные функции. Tellinen определяет форму текущей ветви, используя геометрию мажорной петли и положение точки внутри области гистерезиса, тогда как Madelung определяет топологию магнитной памяти — то есть от какой точки начинается ветвь, к какой предыдущей точке она должна вернуться и в какой момент вложенный цикл должен быть стёрт из истории. В результате магнитную зависимость правильнее записывать не как однозначную функцию $B = B(H)$, а как $B = B(H, \mathcal{R})$, где $\mathcal{R} = \{P_1, P_2, \dots, P_n\}$ представляет сохранённую историю точек разворота. Такой подход особенно удобен для программной реализации. В каждый момент времени необходимо хранить только текущее подтверждённое магнитное состояние, направление изменения поля и стек точек разворота. При очередном развороте новая точка добавляется в стек. При замыкании вложенной петли соответствующая пара точек удаляется. Поэтому модель естественным образом допускает петли первого, второго, третьего и более высоких порядков без введения отдельного набора специальных правил для каждого случая.

Важно подчеркнуть, что комбинация Tellinen + Madelung, рассматриваемая далее, не является готовой стандартной моделью в том виде, в каком она приведена в литературе. В обзоре [1] Tellinen рассматривается как самостоятельная differential scaling model, а правила Madelung — как отдельный принцип history dependence. Здесь они объединяются в единую вычислительную схему: локальная форма ветви наследуется от Tellinen, а точная структура памяти и возврата — от Madelung. Такое объединение позволяет сохранить простоту дифференциальной модели Tellinen и одновременно обеспечить многоуровневую память вложенных минорных петель.

Исходная модель магнитного гистерезиса Tellinen

Будем обозначать верхнюю (upper) и нижнюю (lower) ветви мажорной петли как:

$$C_u(H) \quad (1)$$

$$C_l(H) \quad (2)$$

Ширина области гистерезиса при данном H :

$$D(H) = C_u(H) - C_l(H) \quad (3)$$

Для любой физически допустимой точки внутри мажорной петли должно выполняться условие:
 $C_l(H) \leq B \leq C_u(H)$.

В оригинальной модели Tellinen внутренние ветви задаются непосредственно через дифференциальную проницаемость. При увеличении H :

$$\frac{dB}{dH} = k_{\uparrow}(H, B) \frac{dC_l}{dH} \quad (4)$$

где

$$k_{\uparrow}(H, B) = \frac{C_u(H) - B}{C_u(H) - C_l(H)} \quad (5)$$

При уменьшении H :

$$\frac{dB}{dH} = k_{\downarrow}(H, B) \frac{dC_u}{dH} \quad (6)$$

где

$$k_{\downarrow}(H, B) = \frac{B - C_l(H)}{C_u(H) - C_l(H)} \quad (7)$$

Это базовая конструкция Tellinen. Её привлекательное свойство состоит в том, что коэффициент масштабирования определяется положением текущей точки между обеими ветвями мажорной петли, а не расстоянием до насыщения. Поэтому модель естественнее удерживает возвратные кривые внутри области мажорной петли, чем некоторые более простые масштабируемые модели, например, исходная модель Chan. Физический смысл очень нагляден. Если при увеличении поля мы уже находимся на нижней мажорной кривой $B = C_l(H)$, то $k_{\uparrow} = 1$ и $\frac{dB}{dH} = \frac{dC_l}{dH}$. То есть траектория остаётся на мажорной петле. Если же при увеличении H точка расположена близко к верхней ветви, $B \rightarrow C_u(H)$, то $k_{\uparrow} \rightarrow 0$, и наклон новой возвратной кривой становится мал. Для уменьшающегося поля всё зеркально.

Для решения дифференциальных уравнений удобно ввести новую функцию:

$$y(H, B) = \frac{B - C_l(H)}{C_u(H) - C_l(H)} \quad (8)$$

Тогда всегда $0 \leq y \leq 1$. Причём $y = 0$ соответствует нижней мажорной кривой, а $y = 1$ — верхней. Индукция восстанавливается из y как:

$$B(H) = C_l(H) + yD(H) \quad (9)$$

В этих обозначениях коэффициенты Tellinen становятся особенно простыми:

$$k_{\uparrow} = 1 - y \quad (10)$$

$$k_{\downarrow} = y \quad (11)$$

Теперь можно переписать исходное дифференциальное уравнение Tellinen, используя y . В силу (9) и (3), имеем:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) \quad (12)$$

Тогда для возрастающей петли ($H \uparrow$) получаем:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) = (1 - y) \frac{dC_l}{dH} \quad (13)$$

После сокращений получаем:

$$(H \uparrow) \quad \frac{dy}{dH} = - \frac{y}{D(H)} \frac{dC_u}{dH} \quad (14)$$

Для уменьшающейся ветви ($H \downarrow$) получаем:

$$\frac{dB}{dH} = \frac{dC_l}{dH} + D \frac{dy}{dH} + y \left(\frac{dC_u}{dH} - \frac{dC_l}{dH} \right) = y \frac{dC_u}{dH} \quad (15)$$

После сокращений получаем:

$$(H \downarrow) \quad \frac{dy}{dH} = - \frac{(1-y)}{D(H)} \frac{dC_l}{dH} \quad (16)$$

Эти две формулы чрезвычайно удобны, потому что позволяют получить Tellinen ветви не последовательным Euler-интегрированием, а непосредственно как функцию H .

Пусть возвратная петля начинается в точке $P_a = (H_a, B_a)$. В этой точке:

$$y_a = \frac{B_a - C_l(H_a)}{D(H_a)} \quad (17)$$

Для возрастающей ветви имеем из (14):

$$\frac{dy}{y} = - \frac{1}{D(H)} \frac{dC_u}{dH} dH \quad (18)$$

Интегрируя обе части (18) с начальным условием (17), получаем:

$$y_T(H_{\uparrow}) = y_a \exp \left[- \int_{H_a}^H \frac{1}{D(H)} \frac{dC_u}{dH} dH \right], \quad H > H_a \quad (19)$$

где индекс T означает оригинальную Tellinen ветвь.

Для уменьшающейся ветви удобнее ввести новую функцию:

$$z = 1 - y \quad (20)$$

Тогда получаем из (16):

$$\frac{dz}{z} = \frac{1}{D} \frac{dC_l}{dH} dH, \quad (21)$$

Интегрируя обе части (21), получаем:

$$z_T(H_{\downarrow}) = z_a \exp \left[\int_{H_a}^H \frac{1}{D(H)} \frac{dC_l}{dH} dH \right], H < H_a \quad (22)$$

где $z_a = 1 - y_a$. Поэтому:

$$y_T(H_{\downarrow}) = 1 - (1 - y_a) \exp \left[\int_{H_a}^H \frac{1}{D(H)} \frac{dC_l}{dH} dH \right], H < H_a \quad (23)$$

Включение Madelung магнитной памяти

Стандартная модель Tellinen относится к моделям с локальной памятью: дальнейшее движение определяется текущими значениями H и B и направлением изменения поля, однако последовательность предыдущих точек разворота явно не хранится. Поэтому после нескольких последовательных разворотов обычная Tellinen-модель сама по себе не содержит строгого правила, указывающего, к какой из ранее пройденных точек должна вернуться текущая ветвь. Для введения многоуровневой магнитной памяти воспользуемся правилами Madelung, уже упомянутые во Введении. Их смысл состоит в следующем. Первая возвратная кривая определяется точкой, из которой она вышла. Если на этой кривой происходит новый разворот, следующая ветвь должна вернуться к предыдущей точке разворота. После замыкания вложенного цикла соответствующая история стирается, а движение продолжается по более старой родительской ветви так, как если бы внутреннего цикла не было. Эти свойства соответствуют памяти точки возврата (return-point memory) и правилу стирания памяти (wiping-out memory).

Корневая траектория и структура магнитной памяти

В вычислительной реализации необходимо различать корневую траекторию и стек точек разворота. Корневая траектория $B_{root}(H)$ — это ветвь, к которой система должна вернуться после полного стирания всех вложенных циклов. В рассматриваемой модели возможны три корневых состояния: начальная кривая намагничивания $B_{init}(H)$, верхняя ветвь мажорной петли $C_u(H)$ и нижняя ветвь мажорной петли $C_l(H)$. Удобно хранить их в переменной `root_branch` со значениями “initial”, “upper_major” и “lower_major”: $B_{root}(H) = B_{init}(H)$, $C_u(H)$ или $C_l(H)$, в зависимости от `root_branch`.

Стек `reversal_stack` содержит только реальные точки, в которых направление изменения H действительно сменилось. Виртуальные или вспомогательные точки в стек не помещаются. Если реальные точки разворота обозначить $P_i = (H_i, B_i)$, то магнитное состояние можно представить как совокупность H_{last} , B_{last} , `previous_direction`, `root_branch` и `reversal_stack`. Пустой `reversal_stack` означает отсутствие активной вложенной памяти. В этом случае система движется не по новой Tellinen-ветви, а непосредственно по сохранённой корневой траектории. Следовательно, после полного wiping-out модель восстанавливает именно ту ветвь, на которой находилась до возникновения вложенной структуры.

Первый разворот и возвратная кривая первого порядка

Первый разворот требует отдельного рассмотрения, поскольку в памяти ещё нет предыдущей реальной точки разворота, которая могла бы служить Madelung target point. Независимо от того, является ли корневой траекторией $B_{init}(H)$, $C_u(H)$ или $C_l(H)$, после первого разворота в $P_1 = (H_1, B_1)$ в `reversal_stack` находится только P_1 . Поэтому для возвратной кривой первого порядка применяется исходный закон Tellinen без Madelung-привязки к конечной точке. Иными словами, первая возвратная кривая является обычной Tellinen FORC, начинающейся в P_1 .

Если на FORC происходит следующий разворот, появляется вторая реальная точка P_2 , и с этого момента уже возникает строго определённая target point P_1 . Если же новых разворотов нет, исходная Tellinen FORC по мере изменения поля приближается к соответствующей мажорной ветви: при увеличении H — к $C_l(H)$, при уменьшении H — к $C_u(H)$. При достижении этой ветви с заданной численной точностью точка P_1 удаляется, стек очищается, а соответствующая мажорная ветвь становится новой `root_branch`.

Второй и последующие развороты

После каждого последующего изменения направления подтверждённая точка $P_n = (H_n, B_n)$ добавляется в `reversal_stack`. Начиная со второй реальной точки разворота текущая ветвь имеет однозначно заданные начало и цель: `start=stack[-1]`, `target=stack[-2]`. Иными словами, новая ветвь, вышедшая из последней точки P_n , должна вернуться к непосредственно предшествующей точке P_{n-1} . Именно это правило формирует вложенную структуру памяти Madelung: `Pstart = stack[-1]`, `Ptarget = stack[-2]`, если `len(reversal_stack) ≥ 2`.

Привязка Tellinen-ветви к Madelung return point

Обычная Tellinen-ветвь, запущенная из $P_a = (H_a, B_a)$, в общем случае не обязана точно пройти через сохранённую Madelung target point $P_b = (H_b, B_b)$. Поэтому одной лишь организации стека недостаточно: необходимо сохранить характерную форму Tellinen-ветви, одновременно обеспечив точное выполнение условия возврата. Для этого используется нормировка непосредственно по магнитной индукции B . Пусть $B_T(H)$ — обычная Tellinen-ветвь, вычисленная из P_a по формулам (19) или (23). Обозначим через B_{Tb} значение этой необработанной ветви при $H = H_b$:

$$B_{Tb} = B_T(H_b) \quad (24)$$

Для точного попадания в конечную точку введём коэффициент масштабирования по B :

$$A_B = \frac{B_b - B_a}{B_{Tb} - B_a} \quad (25)$$

Тогда Madelung-ограниченную ветвь задаём как:

$$B_{TM}(H) = B_a + A_B[B_T(H) - B_a] \quad (26)$$

Формулы (24)–(26) образуют предлагаемую нами модификацию исходной Tellinen-модели. Tellinen определяет форму изменения вдоль ветви, а Madelung задаёт точную конечную точку. Если исходная Tellinen-ветвь и без коррекции попадает в target point, то $B_{Tb} = B_b$ и $A_B = 1$, поэтому скорректированная ветвь совпадает с исходной. Важное свойство нормировки непосредственно по B состоит в том, что $\frac{dB_{TM}}{dH} = A_B \frac{dB_T}{dH}$. Поэтому при $A_B > 0$ знак дифференциальной проницаемости исходной Tellinen-ветви сохраняется. В алгоритме неположительное значение A_B рассматривается как недопустимое.

Wiping-out и восстановление parent branch

Когда текущая ветвь достигает P_{target} , расчёт сначала должен попасть точно в сохранённую точку (H_{target}, B_{target}) , а уже затем изменять память. После замыкания соответствующего вложенного цикла две последние точки — P_{start} и P_{target} — удаляются из reversal_stack. Дальнейшее действие определяется не самим фактом удаления, а числом оставшихся реальных точек разворота. Если после удаления в стеке остаются две или более точки, корневая траектория ещё не восстанавливается. Активной становится более старая parent branch, определяемая новой парой stack[-1] и stack[-2]. Если остаётся одна точка, восстанавливается возвратная кривая первого порядка, которая ранее вышла из этой точки. И только если reversal_stack становится полностью пустым, вся вложенная история считается исчерпанной и дальнейшее движение возвращается на root_branch: $\text{len}(\text{stack}) \geq 2 \rightarrow \text{higher-order parent branch}$; $\text{len}(\text{stack}) = 1 \rightarrow \text{FORC}$; $\text{len}(\text{stack}) = 0 \rightarrow \text{root branch}$. Например, для reversal_stack=[P_1, P_2, P_3, P_4] активной является ветвь $P_4 \rightarrow P_3$. После достижения P_3 замкнувшийся внутренний цикл стирается, и стек превращается в [P_1, P_2]. Система продолжает движение по восстановленной parent branch $P_2 \rightarrow P_1$. Только после достижения P_1 и удаления пары [P_1, P_2] стек становится пустым, после чего восстанавливается корневая траектория. Таким образом, наличие ещё хотя бы одной актуальной reversal point означает, что полный возврат к root_branch пока не произошёл.

Правило восстановления возвратной кривой первого порядка одинаково для всех трёх типов root_branch. Если после wiping-out в стеке остаётся единственная точка P_1 , система возвращается к исходной Tellinen FORC, ранее начавшейся в этой точке. Если новых разворотов больше не происходит, эта FORC продолжается до соответствующей мажорной ветви, после чего P_1 удаляется из памяти, стек очищается, а достигнутая мажорная ветвь становится новой root_branch.

Численное вычисление интегралов

В (19) и (23) нам необходимо вычислять интегралы следующего вида:

$$I = \int_a^b f(x) \frac{dg(x)}{dx} dx \quad (27)$$

где $f(x)$ и $g(x)$ — достаточно гладкие функции. Чтобы избежать неоднозначности в обозначениях, будем считать, что N — число интервалов. Тогда используется $N + 1$ узлов: $x_i = a + i\Delta x, i = 0, 1, \dots, N$, с шагом $\Delta x = \frac{b-a}{N}$. Обозначим $f_i = f(x_i)$ и $g_i = g(x_i)$.

Метод трапеций с конечными разностями

Применим к подынтегральной функции $F(x) = f(x)g'(x)$ обычный метод трапеций:

$$I \approx \Delta x \left[\frac{f_0 g'_0}{2} + \sum_{i=1}^{N-1} f_i g'_i + \frac{f_N g'_N}{2} \right] \quad (28)$$

Во внутренних точках производную $g'(x)$ будем вычислять с помощью центральной двусторонней разности $g'_i \approx \frac{g_{i+1} - g_{i-1}}{2\Delta x}, i = 1, \dots, N-1$. На левой границе используется односторонняя разность $g'_0 \approx \frac{g_1 - g_0}{\Delta x}$, а на правой — $g'_N \approx \frac{g_N - g_{N-1}}{\Delta x}$.

После подстановки этих выражений в (28) шаг Δx сокращается, и получаем:

$$I \approx \frac{1}{2} f_0 (g_1 - g_0) + \frac{1}{2} \sum_{i=1}^{N-1} f_i (g_{i+1} - g_{i-1}) + \frac{1}{2} f_N (g_N - g_{N-1}) \quad (29)$$

Это выражение можно существенно упростить. Заметим, что $g_{i+1} - g_{i-1} = (g_i - g_{i-1}) + (g_{i+1} - g_i)$. После группировки членов, относящихся к каждому отдельному приращению $\Delta g_i = g_{i+1} - g_i$, получаем:

$$I \approx \sum_{i=0}^{N-1} \frac{f_i + f_{i+1}}{2} (g_{i+1} - g_i) \quad (30)$$

Для достаточно гладких функций локальная ошибка на одном интервале имеет порядок $O(\Delta x^3)$, а после суммирования по $N \sim \frac{1}{\Delta x}$ интервалам глобальная ошибка составляет $I - I_h = O(\Delta x^2)$. Таким образом, данный метод является методом второго порядка точности.

Более точный метод на основе квадратичной интерполяции Лагранжа

Точность вычисления интеграла можно повысить, если вместо линейной интерполяции использовать на каждом двух соседних интервалах квадратичные полиномы Лагранжа. Рассмотрим три последовательных узла x_i, x_{i+1}, x_{i+2} , причём $x_{i+1} - x_i = x_{i+2} - x_{i+1} = \Delta x$. Введём безразмерную координату $t = \frac{x - x_i}{\Delta x}$. Тогда $0 \leq t \leq 2$, а три узла соответствуют $t = 0, 1, 2$. Полиномы Лагранжа имеют вид

$$L_0(t) = \frac{(t-1)(t-2)}{2} \quad (31)$$

$$L_1(t) = -t(t-2) = t(2-t), \quad (32)$$

$$L_2(t) = \frac{t(t-1)}{2} \quad (33)$$

Аппроксимируем обе функции квадратичными полиномами:

$$f(x) \approx f_i L_0(t) + f_{i+1} L_1(t) + f_{i+2} L_2(t) \quad (34)$$

$$g(x) \approx g_i L_0(t) + g_{i+1} L_1(t) + g_{i+2} L_2(t) \quad (35)$$

На рассматриваемом двойном интервале необходимо вычислить следующий частичный интеграл:

$$I_i = \int_{x_i}^{x_{i+2}} f(x) g'(x) dx \quad (36)$$

Поскольку $x = x_i + t\Delta x$, $dx = \Delta x dt$, и $\frac{dg}{dx} = \frac{1}{\Delta x} \frac{dg}{dt}$, множители Δx сокращаются:

$$I_i = \int_0^2 f(t) \frac{dg(t)}{dt} dt \quad (37)$$

Подставляя (34) и (35) в (36), получаем:

$$I_i \approx \sum_{j=0}^2 \sum_{k=0}^2 f_{i+j} g_{i+k} \int_0^2 L_j(t) L'_k(t) dt \quad (38)$$

Обозначим $A_{jk} = \int_0^2 L_j(t) L'_k(t) dt$. Прямое интегрирование даёт матрицу:

$$\begin{pmatrix} -\frac{1}{2} & \frac{2}{3} & -\frac{1}{6} \\ -\frac{2}{3} & 0 & \frac{2}{3} \\ \frac{1}{6} & -\frac{2}{3} & \frac{1}{2} \end{pmatrix} \quad (39)$$

Получаем:

$$I_i \approx f_i \left(-\frac{1}{2} g_i + \frac{2}{3} g_{i+1} - \frac{1}{6} g_{i+2} \right) + f_{i+1} \left(-\frac{2}{3} g_i + \frac{2}{3} g_{i+2} \right) + f_{i+2} \left(\frac{1}{6} g_i - \frac{2}{3} g_{i+1} + \frac{1}{2} g_{i+2} \right) \quad (40)$$

Это основная трёхточечная формула метода. Если число интервалов N чётное, весь диапазон разбивается на пары интервалов: $[x_0, x_2], [x_2, x_4], \dots, [x_{N-2}, x_N]$. Тогда:

$$I \approx \sum_{i=0}^{N-2} I_i \quad (41)$$

где каждый I_i рассчитывается по (40). Если N нечётно, для последнего оставшегося интервала можно использовать линейную аппроксимацию. На практике ещё проще выбирать чётное N .

Для гладких $f(x)$ и $g(x)$ на двойном интервале длиной $2\Delta x$ разложение в ряд Тейлора показывает, что локальная ошибка начинается с пятой степени шага: $I_i^{(L)} - I_i = O(\Delta x^5)$. После суммирования по приблизительно $\frac{N}{2} \sim \frac{1}{\Delta x}$ двойным интервалам получается глобальная ошибка $I - I_h = O(\Delta x^4)$. Следовательно, квадратичная схема Лагранжа имеет четвёртый порядок точности, тогда как линейная схема — второй.

Алгоритмическая реализация модели Tellinen–Madelung

Ниже приведён полный алгоритм одного подтверждённого шага. Он специально формулируется так, чтобы в дальнейшем тот же state object можно было безопасно использовать и в нелинейном решателе индуктора, где пробные вычисления $B(H)$ должны выполняться без изменения подтверждённой магнитной памяти.

1. Инициализация. Параметры $root_branch$ и H_{start} задаются в Parameters.txt. Допустимы три стартовых варианта: "initial", "upper_major" и "lower_major". Значение B_{start} отдельно не задаётся, а вычисляется автоматически по выбранной корневой траектории: $B_{start} = B_{root}(H_{start})$. Поле H_{start} должно принадлежать общему диапазону верхней и нижней ветвей $H \in [H_{min}, H_{max}]$; это проверяется при считывании параметров. Если задана отдельная начальная кривая и $root_branch="initial"$, дополнительно проверяется её диапазон. Для "upper_major" задаётся $previous_direction="dec"$, для "lower_major" — $previous_direction="inc"$; для начальной кривой направление определяется знаком H_{start} , а при $H_{start} = 0$ принимается $previous_direction=None$. Функции Hscan сохраняются без изменений; сформированная ими история поля сдвигается так, чтобы её первая точка точно совпадала с H_{start} .

2. Проверка изменения поля. Для нового H вычисляется $\Delta H = H - H_{last}$. Если $|\Delta H| < H_{tol}$, поле считается неизменившимся: возвращается Blast, а всё состояние сохраняется без изменений.

3. Определение направления. При $\Delta H > 0$ принимается $direction="inc"$, при $\Delta H < 0$ — $direction="dec"$.

4. Регистрация новой reversal point. Если $previous_direction=None$, новое направление только запоминается. Если $direction$ совпадает с $previous_direction$, новой точки памяти нет. Если направления различаются, подтверждённая точка $P = (H_{last}, B_{last})$ добавляется в $reversal_stack$, после чего $previous_direction$ заменяется новым направлением.

5. Классификация текущей ветви. Дальнейший закон определяется длиной $reversal_stack$. При $len(stack)=0$ движение идёт по $root_branch$; при $len(stack)=1$ активна возвратная кривая первого порядка; при $len(stack) \geq 2$ активна Madelung-ограниченная ветвь более высокого порядка.

6. Пустой стек. Если $reversal_stack=[]$, вычисляется $B = B_{root}(H)$. Для $root_branch="initial"$ это $B_{init}(H)$, для "upper_major" — $C_u(H)$, для "lower_major" — $C_l(H)$. Никакая новая Tellinen-ветвь в этом случае не создаётся.

7. Одна точка в стеке. Пусть $stack=[P_1]$. Предыдущей реальной reversal point ещё нет, поэтому Madelung endpoint constraint применить невозможно. Независимо от типа $root_branch$ строится обычная first-order Tellinen reversal curve из P_1 без endpoint-коррекции Madelung. Если возникает следующий разворот, в $stack$ добавляется P_2 и появляется конкретная target point P_1 .

8. Завершение возвратной кривой первого порядка. Если новых разворотов нет, исходная Tellinen FORC приближается к соответствующей мажорной ветви. При $direction="inc"$ это $C_l(H)$, при $direction="dec"$ — $C_u(H)$. Если разность между FORC и этой мажорной ветвью становится не больше B_{tol} , точка P_1 удаляется, $stack$ очищается, а достигнутая мажорная ветвь становится новой $root_branch$.

9. Две и более точки. При $\text{len}(\text{stack}) \geq 2$ задаются $\text{start} = \text{stack}[-1]$ и $\text{target} = \text{stack}[-2]$. Текущая ветвь вычисляется функцией $\text{tellinen_madelung_branch}(H, \text{start}, \text{target})$, использующей формулы (24)–(26).

10. Проверка достижения target point. Для возрастающей ветви target считается достигнутой или пройденной при $H \geq H_{\text{target}} - H_{\text{tol}}$, а для уменьшающейся — при $H \leq H_{\text{target}} + H_{\text{tol}}$. Пока это условие не выполнено, стек не изменяется.

11. Движение до target. Если target ещё не достигнута, вычисленные H и B становятся текущим результатом шага, но reversal_stack сохраняется неизменным.

12. Закрытие вложенной петли до полного исчерпания памяти. Если target достигнута, траектория сначала устанавливается точно в $(H_{\text{target}}, B_{\text{target}})$. Затем из стека удаляются P_{start} и P_{target} . Если после этого $\text{len}(\text{stack}) \geq 2$, система немедленно восстанавливает более старую parent branch , определяемую новой парой $\text{stack}[-1]$, $\text{stack}[-2]$. Если $\text{len}(\text{stack}) = 1$, восстанавливается соответствующая $\text{first-order parent branch}$. В обоих случаях root_branch ещё не активируется: пока в стеке остаётся хотя бы одна реальная reversal point , вложенная магнитная история ещё не исчерпана.

13. Полное исчерпание reversal points. Если после wiping-out получено $\text{reversal_stack} = []$, дальнейшая траектория немедленно возвращается к сохранённой root_branch . Для исходного размагниченного состояния это $B_{\text{init}}(H)$; для циклов, порожденных на мажорной петле, — соответствующая мажорная ветвь. Именно этот переход реализует полное правило стирания памяти.

14. Прохождение нескольких return points за один шаг. Конечное заданное H может лежать дальше одной или нескольких сохранённых точек возврата. Поэтому обработку crossing следует выполнять циклом while : после каждого закрытия внутренней петли восстанавливается parent branch и снова проверяется, не пройдена ли следующая target point . Один внешний шаг расчёта тем самым может закрыть сразу несколько вложенных циклов.

15. Фиксация подтверждённого состояния. После обработки всех crossings окончательные H и B записываются как H_{last} и B_{last} вместе с обновлёнными $\text{previous_direction}$, root_branch и reversal_stack . Только это состояние используется как исходное для следующего подтверждённого шага.

Для будущего нелинейного расчёта индуктора важно, чтобы функции вычисления ветвей не изменяли переданный state in-place . Каждое пробное значение H_{trial} должно вычисляться из копии подтверждённого состояния; изменения стека и root_branch фиксируются только после принятия соответствующей точки решателем.

Экспериментальные кривые $C_u(H)$ и $C_l(H)$ мажорной петли будут считываться из файла «Major_Loop.csv», где они будут сохранены в виде двух пар колонок: $(H, C_u(H))$, $(H, C_l(H))$, где $H \in [-H_{\text{max}}, H_{\text{max}}]$. Третья пара колонок, если она присутствует, будет содержать $(H, B_{\text{init}}(H))$, где $H \in [-H_{\text{max}}, H_{\text{max}}]$. Если эта пара колонок отсутствует, то кривая начальной намагничённости будет не считываться, а вычисляться как $B_{\text{init}}(H) = \frac{C_u(H) + C_l(H)}{2}$. Несмотря на общий диапазон $H \in [-H_{\text{max}}, H_{\text{max}}]$, число точек в каждой паре колонок может быть разным, поэтому они должны считываться по отдельности. После считывания дискретных значений $C_u(H)$ и $C_l(H)$ будут

создаваться их формосохраняющие кусочно-кубические интерполяционные функции РСНIP, позволяющие вычислять значения в любой точке H . То же самое выполняется для дискретных значений $B_{init}(H)$, если они присутствуют в файле. В противном случае непрерывная функция начальной намагниченности вычисляется через интерполяционные образы $C_u(H)$ и $C_l(H)$ как их среднее. Для тестовых вычислений пары колонок $(H, C_u(H))$, $(H, C_l(H))$ и $(H, B_{init}(H))$ могут быть сформированы из модели Chan.[2]

Структура Python-проекта Tellinen–Madelung

Программная реализация модели на Python построена по той же общей схеме, что и ранее разработанный проект [Chan–Engelhardt](#). [3] Это позволяет сохранить привычную организацию программы и в дальнейшем использовать практически тот же интерфейс при включении модели гистерезиса в нелинейный расчёт индуктора. Основное отличие состоит в том, что аналитические функции мажорной петли Chan заменены табличными зависимостями, считываемыми из файла Major_Loop.csv, а состояние магнитной памяти теперь задаётся корневой ветвью и стеком точек разворота.

Структура проекта, сохраненного в репозитории [GitHub](#), [4] имеет следующий вид:

```
Tellinen-Madelung-Model-BHloops/
|
├── main.py
├── BH.py
├── Hscan.py
├── Parameters.py
├── Parameters.txt
├── Major_Loop.csv
└── BHloop.csv
```

Файл Major_Loop.csv является входным файлом магнитных характеристик материала, а BHloop.csv создаётся программой и содержит рассчитанную траекторию гистерезиса. Вся структура программы в итоге может быть представлена следующей последовательностью:

Parameters.txt → *Parameters.py*

задают начальное состояние и численные параметры;

Major_Loop.csv → *BH.py*

задаёт свойства магнитного материала;

Hscan.py → $H(t)$

задаёт историю возбуждения; далее

main.py → $BH_{\text{increasing/decreasing}}$ → *_advance*

определяет последовательность магнитных состояний;

а внутри `BH.py`

$$\text{Tellinen} + \text{Madelung stack} \rightarrow B(H)$$

формирует текущую ветвь гистерезиса.

Окончательный результат:

$$H(t) \rightarrow B(t)$$

сохраняется как траектория

$$BHloop.csv.$$

По сравнению с проектом [Chan-Engelhardt](#) здесь особенно удачно получилось разделение обязанностей: `Major_Loop.csv` отвечает за свойства материала, `Tellinen` — за локальную форму ветвей, `Madelung stack` — за магнитную память, а `main.py` — только за последовательное продвижение системы по заданной истории $H(t)$. Это же разделение впоследствии позволит практически без изменения магнитного ядра включить модель в нелинейный расчёт индуктора.

`main.py` — основной управляющий модуль

Файл `main.py` выполняет роль верхнего уровня программы. Он не содержит непосредственно математических уравнений `Tellinen` или алгоритма `Madelung`, а организует последовательный расчёт траектории $B(H)$. Из `Parameters.py` считываются начальное магнитное состояние и численные параметры: H_{start} , $root_branch$, H_{tol} .

Переменная:

```
root_branch
```

может принимать три значения:

```
initial
upper_major
lower_major
```

и определяет корневую траекторию, на которой находится материал перед началом вычислений. Начальное значение магнитной индукции не задаётся пользователем отдельно, а вычисляется непосредственно из выбранной кривой: $B_{\text{start}} = B_{\text{root}}(H_{\text{start}})$. Для этого `main.py` вызывает функцию `root_branch_value(Hstart, root_branch)` из модуля `BH.py`. Таким образом, если, например, задано:

```
root_branch=upper_major
Hstart=200
```

то программа автоматически устанавливает $B_{\text{start}} = C_u(200)$. Это исключает возможность задания начальной точки, не принадлежащей выбранной магнитной ветви.

После определения начального состояния `main.py` создаёт временную сетку $t_0, t_1, \dots, t_N$ и вызывает одну из функций файла `Hscan.py`, например:

```
Hscan1(t)
```

или

```
Hscan2(t)
```

Сами функции `Hscan.py` сохранены без изменения относительно проекта [Chan–Engelhardt](#). Чтобы одна и та же сканирующая функция могла использоваться при различных начальных значениях поля, её результат рассматривается как приращение поля относительно начального значения: $H(t) = H_{\text{start}} + H_{\text{scan}}(t) - H_{\text{scan}}(t_0)$. Поэтому независимо от конкретной формы `Hscan`: $H(t_0) = H_{\text{start}}$. Такой подход позволяет изменять исходное магнитное состояние только через `Parameters.txt`, не переписывая тестовые функции возбуждения. После формирования массива `Hvalues` проверяется, что вся требуемая траектория находится внутри диапазона поля, доступного в `Major_Loop.csv`: $H_{\min} \leq H(t) \leq H_{\max}$. При выходе за этот диапазон расчёт прекращается с сообщением об ошибке.

В отличие от `Chan–Engelhardt`, где необходимо было хранить набор параметров $B_d, B_{d,\text{turn}}, H_0, \text{scale}, \text{offset}$, в модели `Tellinen–Madelung` состояние имеет значительно более прямой физический смысл. В каждый момент сохраняются: $H_{\text{last}}, B_{\text{last}}, \text{previous_direction}, \text{root_branch}, \text{reversal_stack}$. Здесь H_{last} и B_{last} — последняя подтверждённая точка траектории.

`previous_direction` принимает значения:

```
"inc"
"dec"
None
```

и показывает направление предыдущего изменения H .

`root_branch` определяет корневую траекторию, которая должна быть восстановлена после полного стирания вложенной магнитной памяти.

`reversal_stack` представляет собой список реальных точек разворота: $\text{reversal_stack} = [P_1, P_2, \dots, P_n]$, где $P_i = (H_i, B_i)$. Именно этот список реализует память `Madelung`.

При старте на мажорной петле направление движения уже физически связано с выбранной ветвью. Для верхней мажорной ветви принимается:

```
previous_direction = "dec"
```

поскольку верхняя ветвь мажорной петли соответствует движению поля в сторону уменьшения. Для нижней:

```
previous_direction = "inc"
```


Если старт производится на начальной кривой $B_{\text{init}}(H)$, направление определяется знаком H_{start} . При $H_{\text{start}} > 0$ предполагается, что система пришла из размагниченного состояния при возрастающем поле, поэтому `previous_direction="inc"`. При $H_{\text{start}} < 0$ используется `previous_direction="dec"`. И только при старте из $H_{\text{start}} = 0, B_{\text{start}} = 0$ направление ещё отсутствует:

```
previous_direction=None
```

Для каждого нового значения H_i программа сравнивает его с предыдущим H_{last} . Если $|H_i - H_{\text{last}}| \leq H_{\text{tol}}$, магнитное поле считается неизменившимся и $B_i = B_{\text{last}}$. Состояние памяти при этом не изменяется. Если $H_i > H_{\text{last}}$, вызывается:

```
BH_increasing(...)
```

из `BH.py`. Если $H_i < H_{\text{last}}$, вызывается:

```
BH_decreasing(...)
```

Обе функции получают текущее подтверждённое состояние: $H_{\text{last}}, B_{\text{last}}, \text{root_branch}, \text{reversal_stack}$ и возвращают $B_i, \text{root_branch}^{\text{new}}, \text{reversal_stack}^{\text{new}}$. После этого $H_{\text{last}} \leftarrow H_i, B_{\text{last}} \leftarrow B_i$, и расчёт переходит к следующему шагу.

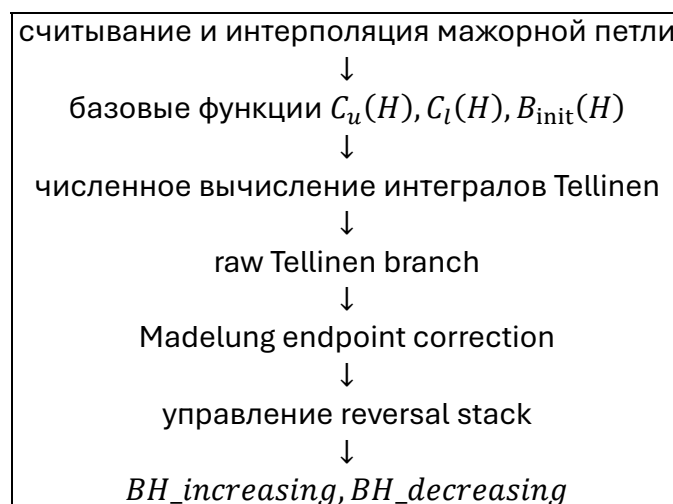
После обработки всего массива формируется

$$BHloop = \begin{pmatrix} H_0 & B_0 \\ H_1 & B_1 \\ \vdots & \vdots \\ H_N & B_N \end{pmatrix},$$

строится график $B(H)$, а значения сохраняются в файл `BHloop.csv` в колонках `H_A_per_m, B_T`.

ВН.py — ядро магнитной модели

Файл `ВН.py` содержит всю физическую и математическую часть алгоритма Tellinen–Madelung. По функциональному назначению он соответствует `ВН.py` проекта [Chan–Engelhardt](#), однако его внутренняя структура существенно отличается. Условно его можно разделить на несколько уровней:



При загрузке ВН.py открывается файл Major_Loop.csv со структурой H_upper, B_upper, H_lower, B_lower, H_init, B_init. Первые две пары колонок являются обязательными: (H_u, B_u) , (H_l, B_l) . Они задают верхнюю и нижнюю ветви мажорной петли. Третья пара (H_{init}, B_{init}) является желательной, если ее можно получить экспериментально. Если такие измерения отсутствуют, то алгоритм попытается восстановить начальную намагниченность из мажорных ветвей. Каждая пара считывается независимо, поэтому число экспериментальных точек на трёх кривых может различаться. Строка заголовков в файле допускается и автоматически пропускается. После чтения значения каждой ветви сортируются по H , и проверяется отсутствие повторяющихся значений поля.

Дискретные данные преобразуются в непрерывные функции посредством [PchipInterpolator](#) библиотеки SciPy. Получаются интерполяционные образы $C_u(H)$, $C_l(H)$, и, при наличии соответствующих данных, $B_{init}(H)$. PCHIP выбран вместо обычного кубического сплайна, поскольку он сохраняет форму монотонной зависимости и значительно меньше склонен создавать искусственные overshoot между экспериментальными точками.

В коде этим зависимостям соответствуют функции:

```
Upper (H)
Lower (H)
Binit (H)
```

Если отдельная начальная кривая в Major_Loop.csv отсутствует, используется $B_{init}(H) = \frac{C_u(H) + C_l(H)}{2}$, которая вычисляется по интерполяционным образам.

Функция:

```
loop_width (H)
```

вычисляет $D(H) = C_u(H) - C_l(H)$. Одновременно выполняется проверка $D(H) > D_{tol}$, поскольку деление на $D(H)$ непосредственно входит в уравнения Tellinen.

Функция:

```
y_from_B (H, B)
```

вычисляет $y = \frac{B - C_l(H)}{C_u(H) - C_l(H)}$. Она одновременно выполняет физическую проверку: $0 \leq y \leq 1$. Если точка существенно выходит за пределы мажорной петли, программа генерирует ошибку. Обратное преобразование выполняется функцией:

```
B_from_y (H, y)
```

по формуле $B = C_l(H) + yD(H)$.

Функция:

```
_integral_f_dg(...)
```

вычисляет интегралы общего вида $I = \int_a^b f(H) dg(H)$. Для Tellinen используются два конкретных интеграла: $I_u(a, b) = \int_a^b \frac{1}{D(H)} dC_u(H)$ и $I_l(a, b) = \int_a^b \frac{1}{D(H)} dC_l(H)$. Им соответствуют функции:

```
integral_upper(a, b)
integral_lower(a, b).
```

Способ интегрирования выбирается параметром IntegrationMethod в Parameters.txt. Допустимы два варианта:

```
trapezoid
lagrange
```

Первый использует разработанную ранее формулу $\approx \sum_i \frac{f_i + f_{i+1}}{2} (g_{i+1} - g_i)$ со вторым порядком точности. Второй использует трёхточечную квадратичную интерполяцию Лагранжа с глобальной погрешностью порядка $O(\Delta H^4)$. Число интервалов задаётся параметром IntegrationIntervals.

Функция:

```
tellinen_raw_y(H, start, direction)
```

реализует аналитически интегрированные уравнения Tellinen. Пусть $start = (H_a, B_a)$. Сначала определяется $y_a = \frac{B_a - C_l(H_a)}{D(H_a)}$. При возрастающем поле $direction = "inc"$ вычисляется $y_T(H) = y_a \exp \left[- \int_{H_a}^H \frac{dC_u(h)}{D(h)} \right]$. При уменьшающемся поле $direction = "dec"$ используется $y_T(H) = 1 - (1 - y_a) \exp \left[\int_{H_a}^H \frac{dC_l(h)}{D(h)} \right]$.

Функция:

```
tellinen_raw_branch(...)
```

преобразует полученное значение y_T обратно в магнитную индукцию: $B_T(H) = C_l(H) + y_T(H)D(H)$. Таким образом, эта функция строит чистую исходную Tellinen-ветвь без памяти Madelung.

Если имеются две или более реальные точки разворота, последняя из них является началом текущей ветви, $P_a = (H_a, B_a)$, а предыдущая — $P_b = (H_b, B_b)$ — Madelung return point.

Функция:

```
tellinen_madelung_branch(
    H,
    start,
    target,
    direction
)
```

сначала вычисляет обычную Tellinen-ветвь $B_T(H)$ и её значение в целевой точке $B_{Tb} = B_T(H_b)$. После этого вычисляется коэффициент масштабирования $A_B = \frac{B_b - B_a}{B_{Tb} - B_a}$. Окончательная ветвь определяется как $B_{TM}(H) = B_a + A_B[B_T(H) - B_a]$. Таким образом, форма ветви наследуется от Tellinen, но она точно проходит через Madelung return point: $B_{TM}(H_a) = B_a$, $B_{TM}(H_b) = B_b$. В коде дополнительно проверяется условие $A_B > 0$. Это принципиально важно, поскольку $\frac{dB_{TM}}{dH} = A_B \frac{dB_T}{dH}$ и положительный коэффициент сохраняет направление исходной Tellinen-ветви. После вычисления также контролируется условие $C_l(H) \leq B_{TM}(H) \leq C_u(H)$.

Функция:

```
_advance(
    H,
    direction,
    root_branch,
    reversal_stack
)
```

является центральной частью реализации Madelung memory. Именно она анализирует число сохранённых reversal points и выбирает, какую ветвь необходимо использовать. При этом входной reversal_stack никогда не изменяется непосредственно:

```
stack = list(reversal_stack)
```

Создаётся его копия. Такое устройство особенно важно для будущего nonlinear solver, где одно и то же подтверждённое состояние должно многократно использоваться при пробных значениях H . Если стек пуст, $reversal_stack = []$, вложенной магнитной памяти нет. Тогда $B = B_{root}(H)$. В зависимости от root_branch это: $B_{init}(H)$, $C_u(H)$, $C_l(H)$. Если в стеке имеется одна точка $[P_1]$, это первая возвратная кривая — FORC. Предыдущей реальной reversal point ещё нет, поэтому Madelung target не определена. Следовательно, используется непосредственно исходная Tellinen-ветвь $B = B_T(H; P_1)$. Это правило применяется независимо от того, находилась P_1 на начальной кривой или на мажорной ветви. При продолжительном движении такая Tellinen FORC постепенно приближается к противоположной мажорной ветви. При возрастающем H : $B_T(H) \rightarrow C_l(H)$, а при уменьшающемся: $B_T(H) \rightarrow C_u(H)$. Когда расстояние становится меньше B_{tol} , первая точка разворота считается исчерпанной, стек очищается, а соответствующая мажорная ветвь становится новой корневой: lower_major или upper_major. При наличии двух или более точек текущая ветвь однозначно определяется последними двумя элементами: $start = stack[-1]$, $target = stack[-2]$. Сначала проверяется геометрическая согласованность. Для

возрастающей ветви необходимо $H_{\text{target}} > H_{\text{start}}$, а для уменьшающейся $H_{\text{target}} < H_{\text{start}}$. До достижения target значение B вычисляется функцией:

```
tellinen_madelung_branch(...)
```

Если очередное заданное значение H достигло или прошло сохранённую target point, соответствующая вложенная петля считается замкнутой. Из стека удаляются две последние точки:

```
stack.pop()
stack.pop()
```

то есть P_{start} и P_{target} . После этого функция не завершает работу, а возвращается к началу цикла while. Это позволяет автоматически определить восстановленную parent branch. Если после удаления осталось $\text{len}(\text{stack}) \geq 2$, восстанавливается более старая Tellinen–Madelung ветвь. Если $\text{len}(\text{stack}) = 1$, восстанавливается FORC. Если $\text{len}(\text{stack}) = 0$, восстанавливается root branch. Тем самым один большой численный шаг поля способен последовательно закрыть несколько вложенных циклов. Схематически: $[P_1, P_2, P_3, P_4] \rightarrow [P_1, P_2] \rightarrow []$ с последовательным восстановлением $P_4 \rightarrow P_3$, затем $P_2 \rightarrow P_1$, и наконец корневой ветви.

Функции BH_increasing() и BH_decreasing() образуют внешний интерфейс BH.py, аналогично одноимённым функциям проекта [Chan–Engelhardt](#). Для возрастающего поля вызывается:

```
BH_increasing(...)
```

а для уменьшающегося:

```
BH_decreasing(...)
```

Если направление движения не изменилось, текущий стек передаётся функции _advance() без изменений. Если произошёл разворот, например движение было уменьшающимся, а стало возрастающим, последняя подтверждённая точка $(H_{\text{last}}, B_{\text{last}})$ добавляется в стек:

```
stack_new.append(
    (Hlast, Blast)
)
```

После этого вызывается _advance() уже для нового направления. Таким образом, BH_increasing() и BH_decreasing() выполняют только две операции:

1. обнаруживают и регистрируют reversal point
2. передают расчёт текущей ветви центральной функции _advance()

Это делает структуру BH.py существенно проще и прозрачнее, чем в Chan–Engelhardt.

Parameters.py

Файл Parameters.py считывает файл Parameters.txt и преобразует значения в числовые или строковые параметры. Он должен работать не только с числами, поскольку имеются строковые параметры:

```
root_branch
IntegrationMethod
```

После чтения выполняется проверка обязательных параметров и допустимых значений. Например, root_branch может иметь только значения:

```
initial
upper_major
lower_major
```

a IntegrationMethod:

```
trapezoid
lagrange
```

Проверяется также положительность $H_{tol}, B_{tol}, D_{tol}$ и корректность IntegrationIntervals. Если Major_Loop.csv уже существует, Parameters.py дополнительно определяет допустимый общий диапазон верхней и нижней мажорных ветвей и проверяет: $H_{min} \leq H_{start} \leq H_{max}$. Для root_branch=initial при наличии отдельной экспериментальной начальной кривой проверяется также принадлежность H_{start} диапазону этой кривой.

Parameters.txt

Файл предназначен для изменения параметров расчёта без редактирования Python-кода. В текущем варианте он содержит четыре группы данных. Первая — параметр:

H_c

который пока сохраняется исключительно потому, что неизменённая функция Hscan2() использует его при построении тестового сигнала. Вторая группа определяет начальное магнитное состояние:

```
root_branch=initial
Hstart=0.0
```

Третья задаёт численные допуски:

```
H_tol
B_tol
D_tol
```

Четвёртая управляет вычислением интегралов:

```
IntegrationMethod
IntegrationIntervals
```

Таким образом, параметры самого материала B_s, B_r, H_c больше не являются параметрами модели Tellinen. Геометрия реального материала поступает непосредственно из Major_Loop.csv.

Hscan.py

Файл Hscan.py сохранён практически без изменений относительно проекта [Chan–Engelhardt](#). Функция:

```
Hscan1(t)
```

создаёт затухающую синусоидальную зависимость поля и удобна для генерации большого числа последовательно уменьшающихся симметричных минорных циклов.

Функция:

```
Hscan2(t)
```

создаёт затухающую осцилляцию на фоне смещённого магнитного поля и предназначена для проверки асимметричных вложенных петель. Сохранение тех же функций позволяет непосредственно сравнивать результаты Chan–Engelhardt и Tellinen–Madelung при одной и той же истории внешнего поля.

Major_Loop.csv

Этот файл принципиально отличает новый проект от [Chan–Engelhardt](#). В Chan форма мажорной петли вычислялась аналитически из параметров B_s, B_r, H_c . В Tellinen–Madelung исходная геометрия задаётся таблицами:

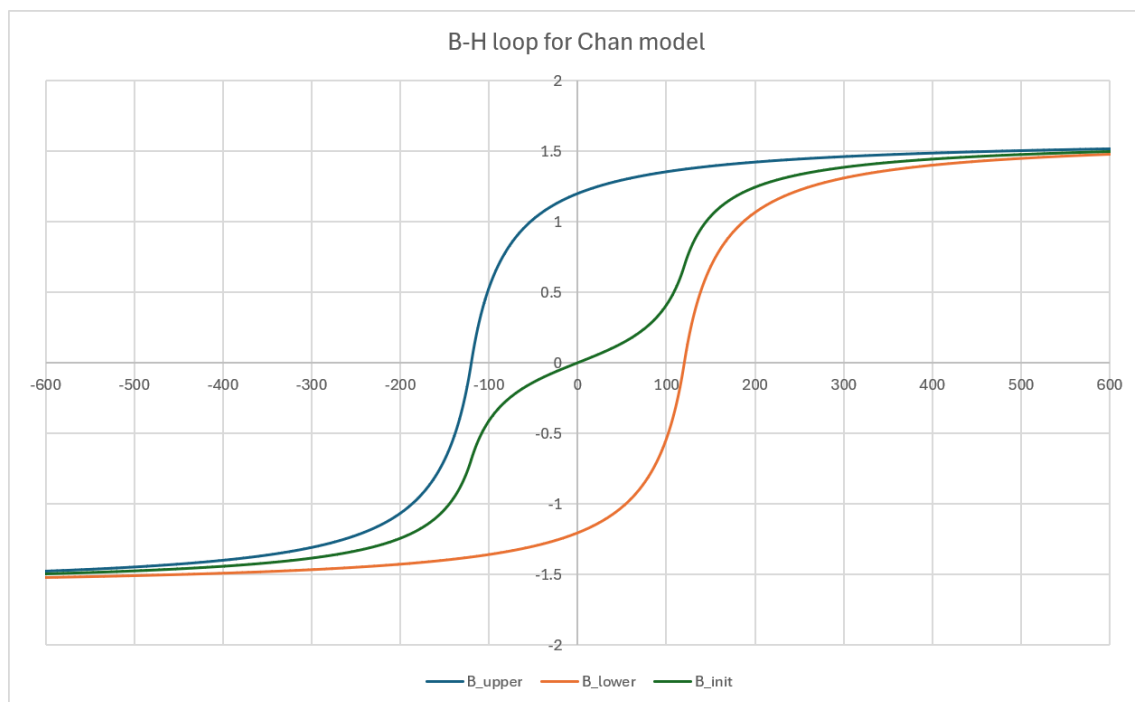
```
H_upper, B_upper,  
H_lower, B_lower,  
H_init, B_init
```

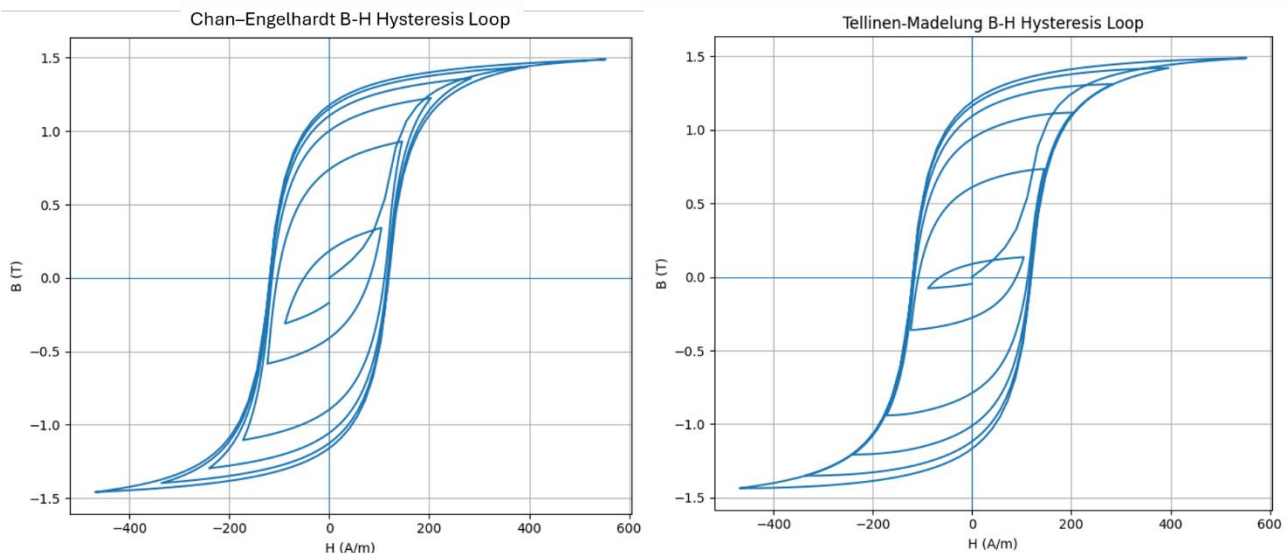
Это позволяет использовать непосредственно экспериментально измеренные кривые. Количество точек в каждой паре колонок может различаться. Программа независимо считывает каждую ветвь, после чего создаёт её непрерывный PCNIP-интерполяционный образ. Для отладки программы Major_Loop.csv может быть сгенерирован из аналитической модели Chan. В дальнейшем тот же формат позволяет заменить тестовую петлю реальными экспериментальными данными без изменения алгоритма Tellinen–Madelung.

Сравнение расчетов по моделям Tellinen–Madelung и Chan–Engelhardt

Для сравнения минорных петель в моделях Tellinen–Madelung и Chan–Engelhardt мы сгенерировали тестовую мажорную петлю по модели Chan [2] со следующими параметрами:

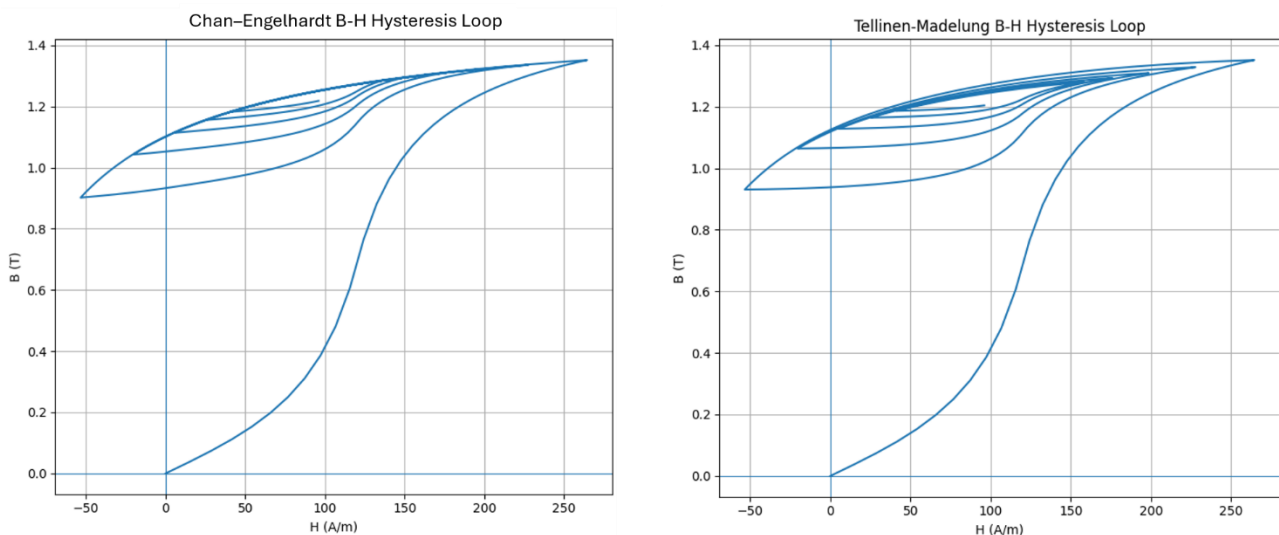
```
# Saturation magnetic flux density, T  
Bs = 1.6  
  
# Remanent magnetic flux density, T  
Br = 1.2  
  
# Coercive field strength, A/m  
Hc = 120.0  
  
# Maximum absolute field in the generated table, A/m  
Hmax = 600.0
```





$$H_{scan1}(t) = 600 \sin(120\pi t) \exp[-t/(3T)], \quad T = 2\pi/(120\pi) = 1/60 \text{ s}$$

Синусоидальное поле 60 Hz с начальной амплитудой 600 A/m и экспоненциальной постоянной затухания $3T = 0.05 \text{ s}$. Такой сигнал естественно создаёт последовательность уменьшающихся по амплитуде петель вокруг нуля.



$$H_{scan2}(t) = H_{bias}(1 - e^{-(\tau/\tau_{bias})}) + A_0 e^{-(\tau/\tau_{decay})} \sin(\omega\tau)$$

В текущем коде $H_{bias} = 0.8 \cdot H_c$, $A_0 = 1.5 \cdot H_c$, $n_{cycles} = 5$, $\tau_{bias} = 0.01 \cdot T_{sim}$, $\tau_{decay} = 0.8 \cdot T_{sim}$, где T_{sim} — время симуляций. Цель H_{scan2} — создать несколько разворотов вокруг положительного смещения.

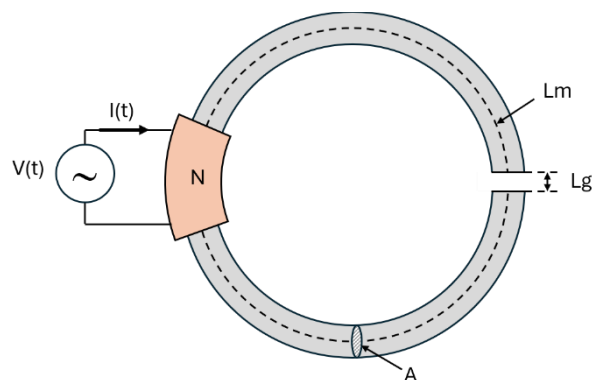
В целом поведения в обоих моделях похожие, однако можно заметить, что минорные ветви в модели Tellinen-Madelung более пологие. Это предопределено уже в самих уравнениях (4)-(7), где наклон минорных кривых определяется удалением от мажорных. В модели Chan-Engelhardt этот механизм отсутствует. Влияние магнитной памяти проследить сложнее. Возможно, для демонстрации этого эффекта могут понадобиться специфические возбуждения. Честно говоря, какие-либо радикальные отличия для двух моделей очень насторожили бы нас, поскольку это существенно осложнило бы выбор модели гистерезиса при описании реального материала.

Модель индуктора с магнитным сердечником

В качестве первого практического применения модели [Tellinen–Madelung](#) [4] рассмотрим влияние гистерезиса и поведения минорной петли на переходные процессы в индукторе с ферромагнитным сердечником. Для оценки получаемых результатов проведём сравнение с расчётами в [LTspice](#), где реализована модель [Chan–Engelhardt](#). [3] Такое сопоставление позволит оценить, насколько модель Tellinen–Madelung пригодна для решения практических задач схемотехнического моделирования. Расчёты LTspice будем использовать в качестве референсных, поскольку этот симулятор широко применяется в инженерной практике и его нелинейные магнитные модели хорошо отработаны в задачах анализа электрических цепей. Вместе с тем результаты LTspice не следует рассматривать как безусловный физический эталон. Известно, что используемые в нём модели имеют ограничения, которые могут становиться существенными при быстрых процессах намагничивания, а также для некоторых типов магнитных материалов.[5] Поэтому сравнение должно служить прежде всего средством взаимной проверки моделей и оценки области их применимости, а не безоговорочным подтверждением одной модели результатами другой.

Основы теории магнитных цепей предполагаются известными, поэтому здесь приводятся только те положения, которые непосредственно необходимы для построения расчетного алгоритма.

Геометрия индуктора показана ниже. Форма сердечника и его однородное поперечное сечение могут быть произвольными. Мы выбрали кольцо. Также допускается наличие щели в сердечнике.



Запишем уравнения для напряженностей магнитного поля и тока в катушке $I(t)$, которые связаны между собой:

$$\begin{cases} H_m(t)L_m + H_g(t)L_g = NI(t) \\ RI(t) = V(t) - NA \frac{dB(t)}{dt} \end{cases} \quad (42)$$

где H_m – напряженность магнитного поля (A/m) внутри магнитного сердечника, H_g – напряженность магнитного в щели (A/m), R – сопротивление катушки (Ω), $V(t)$ – внешний источник эдс (V), и $-NA \frac{dB(t)}{dt}$ – эдс самоиндукции катушки. При одинаковом сечении сердечника магнитная индукция $B(t)$ будет равномерно распределенной вдоль него в силу условия

непрерывности ее нормальной компоненты на границах раздела. Однако необходимо учесть ее зависимость от напряженности магнитного поля. В качестве независимого магнитного параметра выберем $H_m(t)$, тогда $B(t) = B(H_m(t))$. В пределах щели можно сразу записать $H_g(t) = B(H_m(t))/\mu_0$. В результате получаем следующую систему уравнений:

$$\begin{cases} I(t) = \frac{L_m}{N} H_m(t) + \frac{L_g}{\mu_0 N} B(H_m(t)) \\ RI(t) = V(t) - NA \frac{dB(H_m(t))}{dt} \end{cases} \quad (43)$$

Подставляя ток из первого уравнения во второе, получаем нелинейное дифференциальное уравнение относительно напряженности магнитного поля, замыкаемое гистерезисной зависимостью $B(H_m(t))$:

$$NA \frac{dB(H_m(t))}{dt} + \frac{RL_m}{N} H_m(t) + \frac{RL_g}{\mu_0 N} B(H_m(t)) = V(t) \quad (44)$$

Для численного решения (44) воспользуемся методом конечных разностей [Backward Euler](#) для временного шага $t_{n+1} = t_n + \Delta t$:

$$NA \frac{B(H_{m,n+1}) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H_{m,n+1} + \frac{RL_g}{\mu_0 N} B(H_{m,n+1}) = V_{n+1} \quad (45)$$

где $H_{m,n} = H_m(t_n)$, $H_{m,n+1} = H_m(t_n + \Delta t)$, и $V_{n+1} = V(t_n + \Delta t)$.

Для численного решения (45) вводится функция невязки:

$$F(H) = NA \frac{B(H) - B(H_{m,n})}{\Delta t} + \frac{RL_m}{N} H + \frac{RL_g}{\mu_0 N} B(H) - V_{n+1}. \quad (46)$$

Искомое значение магнитного поля определяется из условия $F(H_{m,n+1}) = 0$. После нахождения $H_{m,n+1}$ на очередном шаге вычисляются магнитная индукция и ток:

$$B_{n+1} = B(H_{m,n+1}) \quad (47)$$

$$I_{n+1} = \frac{L_m}{N} H_{m,n+1} + \frac{L_g}{\mu_0 N} B(H_{m,n+1}) \quad (48)$$

Особенность задачи состоит в том, что вычисление $B(H)$ связано с моделью гистерезиса, обладающей памятью. Состояние модели на момент t_n можно условно обозначить через S_n . Оно включает текущее положение на петле гистерезиса, направление предыдущего движения, и точки разворота. Во время решения нелинейного уравнения по невязке $F(H)$ состояние S_n должно оставаться неизменным. Каждое пробное значение поля H_{trial} рассматривается как независимая гипотеза о состоянии системы в момент t_{n+1} . Для каждого такого значения выполняется полный алгоритм гистерезиса: $(H_{\text{trial}}, S_n) \rightarrow (B_{\text{trial}}, S_{\text{trial}})$. При этом модель может продолжить текущую ветвь гистерезиса либо определить, что между шагами n и $n+1$ произошёл разворот, и перейти на соответствующую новую ветвь. Все переходы выполняются точно так же, как при обычном последовательном расчёте гистерезисной траектории. Однако полученное состояние S_{trial} является только пробным. Оно не заменяет состояние S_n , пока соответствующее значение H_{trial} не удовлетворит нелинейному уравнению. Следующее пробное значение поля снова рассчитывается исходя из того же самого состояния S_n : $(H_{\text{trial},1}, S_n) \rightarrow$

$(B_{\text{trial},1}, S_{\text{trial},1}), (H_{\text{trial},2}, S_n) \rightarrow (B_{\text{trial},2}, S_{\text{trial},2})$, и так далее. Нельзя использовать состояние, полученное на одной пробной итерации, в качестве исходного состояния для следующей пробной итерации. В противном случае последовательность точек, генерируемая методом поиска корня, сама стала бы искусственной магнитной предысторией и изменила бы петлю гистерезиса. Таким образом, при поиске корня выполняется полный алгоритм модели гистерезиса, но все возвращаемые ею новые параметры сохраняются как пробные. Только после нахождения окончательного значения $H_{m,n+1}$ соответствующее пробное состояние принимается как новое физическое состояние системы: $S_{n+1} = S_{\text{trial}}$.

Для применения [метода бисекции](#) при поиске корня нелинейного уравнения $F(H) = 0$ необходимо найти интервал $[H_a, H_b]$, на концах которого функция невязки имеет разные знаки: $F(H_a)F(H_b) \leq 0$. В качестве первой границы естественно принять известное значение поля на предыдущем временном шаге: $H_a = H_{m,n}$. При этом $B(H_a) = B(H_{m,n})$, поэтому производный член в функции невязки обращается в нуль. Следовательно, $F(H_{m,n}) = RI_n - V_{n+1}$. Величина $V_{n+1} - RI_n$ определяет физически ожидаемое направление изменения магнитной индукции. Действительно, из электрического уравнения $NA \frac{dB}{dt} = V - RI$ следует, что при $V_{n+1} - RI_n > 0$ магнитная индукция имеет тенденцию возрастать, а при $V_{n+1} - RI_n < 0$ — уменьшаться. Поэтому сначала поиск второй границы интервала проводится в соответствующем направлении от $H_{m,n}$. Задаётся начальный шаг поиска ΔH_0 . Первая пробная граница выбирается как $H_b = H_{m,n} + s\Delta H_0$, где:

$$s = \begin{cases} +1, & V_{n+1} - RI_n > 0, \\ -1, & V_{n+1} - RI_n < 0. \end{cases}$$

В точке H_b вычисляется значение $B(H_b)$ при сохранённом состоянии S_n , после чего определяется $F(H_b)$. Если выполняется условие $F(H_a)F(H_b) \leq 0$, то корень заключён в найденном интервале и можно переходить к методу бисекции для его поиска с предустановленной точностью и ограничением на максимальное число шагов. Если знаки невязки на концах одинаковы, расстояние от $H_{m,n}$ до второй границы увеличивается в два раза: $\Delta H_0, 2\Delta H_0, 4\Delta H_0, 8\Delta H_0, \dots$ Таким образом, $H_b^{(j)} = H_{m,n} + s 2^j \Delta H_0$. На каждом таком пробном значении снова выполняется независимый расчёт гистерезиса из одного и того же сохранённого состояния S_n . Если в физически ожидаемом направлении интервал со сменой знака найти не удалось, для повышения надёжности расчёта поиск повторяется в противоположном направлении. При этом используется тот же алгоритм последовательного удвоения расстояния от $H_{m,n}$. Такой дополнительный поиск особенно полезен около точек разворота петли гистерезиса, где простая оценка направления по величине $V_{n+1} - RI_n$ может оказаться недостаточной.

После нахождения интервала $[H_a, H_b]$, удовлетворяющего условию $F(H_a)F(H_b) \leq 0$, начинается собственно [метод бисекции](#). На каждой итерации вычисляется середина интервала: $H_c = \frac{H_a + H_b}{2}$. Для этого пробного значения, начиная из неизменного состояния S_n , выполняется полный расчёт модели гистерезиса и определяется $B_c = B(H_c)$, после чего вычисляется невязка $F_c = F(H_c)$. Если $F(H_a)F(H_c) \leq 0$, то корень находится в интервале $[H_a, H_c]$, и принимается $H_b = H_c$. Если же $F(H_a)F(H_c) > 0$, то корень находится в интервале $[H_c, H_b]$, и принимается $H_a = H_c$.

После этого процедура повторяется. Поиск завершается, когда выполняется одно из условий сходимости: $|F(H_c)| \leq \varepsilon_F$, или $|H_b - H_a| \leq \varepsilon_H$, где ε_F — допустимая невязка нелинейного уравнения, а ε_H — допустимая ширина интервала поиска по напряжённости магнитного поля. Если за заданное максимальное число итераций необходимая точность не достигнута, временной шаг считается нерешённым. В этом случае следует уменьшить Δt , изменить параметры поиска интервала либо проверить параметры магнитной и электрической моделей. Не следует автоматически принимать последнее приближение, поскольку оно может не удовлетворять исходному уравнению с требуемой точностью.

После нахождения корня принимается $H_{m,n+1} = H_c$. Одновременно с этим принимаются соответствующие этому корню значения (47) и (48), а пробное состояние модели гистерезиса становится новым подтверждённым состоянием: $S_{n+1} = S_{\text{trial}}$. Только после этого временной шаг считается завершённым и состояние $H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1}$ используется как исходное при переходе к следующему моменту времени. В результате весь алгоритм можно представить в виде следующей последовательности:

$$\boxed{(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{поиск интервала} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1})}.$$

Принципиальной особенностью метода является то, что все пробные расчёты внутри поиска корня выполняются при неизменном исходном состоянии гистерезиса S_n , а новое состояние фиксируется только после окончательного решения нелинейного уравнения.

Алгоритмическая реализация расчёта индуктора

Весь алгоритм можно представить в виде следующей последовательности:

$$\boxed{(H_{m,n}, B_n, I_n, S_n) \rightarrow V_{n+1} \rightarrow \text{поиск интервала} \rightarrow F(H_{m,n+1}) = 0 \rightarrow (H_{m,n+1}, B_{n+1}, I_{n+1}, S_{n+1})}.$$

- 1. Подтверждённое состояние в начале шага.** В момент t_n известны H_n, B_n, I_n и магнитное состояние S_n , включающее `previous_direction`, `root_branch` и `reversal_stack`. Для следующего момента вычисляется $V_{n+1} = V(t_{n+1})$.
- 2. Пробное значение поля.** Любое H_{trial} рассматривается только как гипотеза о состоянии системы в момент t_{n+1} . Для него из копии подтверждённого состояния S_n выполняется полный расчёт Tellinen–Madelung и получаются B_{trial} и пробное состояние S_{trial} .
- 3. Пробный ток и невязка.** По уравнению магнитной цепи вычисляется I_{trial} , после чего по (46) определяется $F(H_{\text{trial}})$. Таким образом, каждому H_{trial} соответствует согласованный набор $H_{\text{trial}}, B_{\text{trial}}, I_{\text{trial}}, F(H_{\text{trial}}), S_{\text{trial}}$.
- 4. Выбор первого направления поиска.** По знаку $\text{drive} = V_{n+1} - RI_n$ выбирается физически ожидаемое направление изменения поля. В этой точке определяется только порядок поиска; окончательное направление по-прежнему устанавливается самим решением нелинейного уравнения.

5. Поиск интервала для бисекции. В выбранном направлении выполняется расширяющийся поиск с шагом initial_H_step и множителем $\text{bracket_growth_factor}$. Смена знака F контролируется между соседними пробными точками. Все пробные значения вычисляются из одного и того же состояния S_n .

6. Дополнительное сканирование и обратное направление. Если расширяющийся поиск не дал bracket , допустимый интервал H сканируется более плотно с числом разбиений $\text{bracket_scan_points}$. При неудаче та же процедура повторяется в противоположном направлении. Если $F(H_a)F(H_b) \leq 0$ получить не удалось, временной шаг считается нерешённым.

7. Итерация бисекции. Для найденного интервала вычисляется $H_m = \frac{H_a + H_b}{2}$. Значения B_m, I_m и пробное магнитное состояние S_m снова рассчитываются непосредственно из S_n , после чего определяется $F(H_m)$.

8. Сужение интервала. По знакам $F(H_a), F(H_m), F(H_b)$ сохраняется та половина интервала, в которой остаётся смена знака невязки. Затем вычисляется новая середина и процедура повторяется.

9. Критерий сходимости. Итерационный поиск прекращается, когда выполняется хотя бы одно из условий: $|F(H_m)| \leq \text{residual_tol}$ или $|H_b - H_a| \leq H_{\text{tol}}$. Если за $\text{max_bisection_iter}$ требуемая точность не достигнута, последнее приближение автоматически не принимается.

10. Принятие решения. После нахождения корня принимаются H_{n+1} , соответствующие ему B_{n+1} и I_{n+1} , а также именно то пробное состояние S_{n+1} , которое было получено для принятого значения поля.

11. Фиксация магнитной памяти. Только после принятия корня root_branch , reversal_stack и $\text{previous_direction}$ заменяются их новыми значениями. Все остальные trial_state , возникавшие при поиске интервала и в процессе бисекции, отбрасываются.

12. Переход к следующему временному шагу. Принятые $H_{n+1}, B_{n+1}, I_{n+1}$ и S_{n+1} становятся единственным исходным состоянием для расчёта момента t_{n+2} . Тем самым внутренняя последовательность пробных точек итерационного решателя не превращается в искусственную магнитную предысторию.

Принципиальной особенностью метода является то, что все пробные расчёты внутри поиска корня выполняются при неизменном исходном состоянии гистерезиса S_n , а новое состояние фиксируется только после окончательного решения нелинейного уравнения.

Структура Python-проекта для расчёта индуктора

После построения модели гистерезиса следующий проект использует её как магнитное ядро при решении нелинейной задачи для индуктора. Теоретическая постановка этой задачи приведена выше в уравнениях (42)–(48). В программной реализации напряжённость магнитного поля H уже не задаётся заранее функцией $H_{scan}(t)$, а определяется на каждом временном шаге из электрического уравнения совместно с гистерезисной зависимостью $B(H)$.

Структура проекта имеет следующий вид:

```
Tellinen-Madelung-Model-Inductor/  
├── main.py  
├── BH.py  
├── Parameters.py  
├── Parameters.txt  
├── V.py  
├── Major_Loop.csv  
  
├── Current.csv  
├── BHloop.csv  
└── State.csv
```

Файл `BH.py` содержит уже описанную ранее модель Tellinen–Madelung и в данном проекте выполняет роль самостоятельного магнитного модуля. Важным свойством его реализации является то, что переданный `reversal_stack` не изменяется непосредственно: перед расчётом создаётся его копия. Поэтому одно и то же подтверждённое магнитное состояние можно многократно использовать при пробных значениях H , возникающих в процессе решения нелинейного уравнения. Внешним интерфейсом магнитной модели остаются функции `BH_increasing()` и `BH_decreasing()`, которые получают подтверждённые H_{last} , B_{last} , `root_branch` и `reversal_stack` и возвращают рассчитанную индукцию вместе с новым пробным состоянием памяти. Таким образом, основное отличие нового проекта от программы непосредственного построения $B(H)$ состоит в функции `main.py`. Ранее последовательность значений H задавалась извне, а магнитная модель только вычисляла соответствующие значения B . В задаче индуктора на каждом временном шаге неизвестным является H_{n+1} , которое определяется из нелинейного уравнения методом бисекции. Сам модуль `BH.py` при этом используется практически как независимая функция с памятью: $(H_{trial}, S_n) \rightarrow (B_{trial}, S_{trial})$, где S_n обозначает подтверждённое магнитное состояние в начале очередного временного шага.

main.py — основной модуль расчёта индуктора

В начале main.py формируется временная сетка. Число шагов определяется отношением $\text{simulation_time}/dt$, причём программа дополнительно проверяет, что полное время расчёта является целым числом временных шагов. После этого задаётся исходное магнитное состояние. Значение H_0 берётся из параметра Hstart, а root_branch определяет, на какой из трёх возможных корневых траекторий находится материал: initial, upper_major или lower_major. Начальная индукция не задаётся независимо, а вычисляется функцией:

```
root_branch_value(Hlast, root_branch)
```

Начальный ток затем определяется непосредственно из уравнения магнитной цепи: $I_0 = \frac{L_m H_0 + (L_g/\mu_0) B_0}{N}$. Поэтому начальные H_0 , B_0 и I_0 всегда согласованы между собой. Одновременно создаётся пустой reversal_stack, а значение previous_direction устанавливается в соответствии с выбранной корневой ветвью и начальным полем.

Основная особенность программы связана с тем, что в течение одного временного шага модель гистерезиса вызывается многократно. Для этого используется функция:

```
evaluate_hysteresis_trial(Htrial)
```

Она рассматривает H_{trial} как пробное значение поля в момент t_{n+1} . Если оно практически совпадает с подтверждённым H_{last} , возвращаются B_{last} и копия текущего состояния памяти. При $H_{\text{trial}} > H_{\text{last}}$ вызывается BH_increasing(), а при $H_{\text{trial}} < H_{\text{last}}$ — BH_decreasing(). При этом обе функции всегда получают одно и то же подтверждённое состояние предыдущего временного шага. Полученные root_trial и stack_trial рассматриваются исключительно как пробные и ещё не изменяют физическую магнитную историю.

Для каждого пробного значения поля функция residual() сначала получает B_{trial} и пробное магнитное состояние, а затем вычисляет ток: $I_{\text{trial}} = \frac{L_m H_{\text{trial}} + (L_g/\mu_0) B_{\text{trial}}}{N}$. После этого рассчитывается невязка дискретизированного электрического уравнения: $F(H_{\text{trial}}) = RI_{\text{trial}} + NA \frac{B_{\text{trial}} - B_{\text{last}}}{\Delta t} - V_{\text{next}}$. Таким образом, каждому пробному значению H_{trial} соответствует полный согласованный набор: $H_{\text{trial}}, B_{\text{trial}}, I_{\text{trial}}, S_{\text{trial}}, F(H_{\text{trial}})$. Именно функция residual() связывает электрическую часть задачи с моделью магнитного гистерезиса.

Поиск интервала для метода бисекции

Перед применением метода бисекции необходимо найти значения H_a и H_b , для которых: $F(H_a)F(H_b) \leq 0$. Эта операция вынесена в отдельную функцию:

```
_find_bracket_in_direction()
```

В качестве исходной точки всегда используется подтверждённое значение H_{last} . Поиск первоначально проводится только в одном направлении. Если это возрастающее направление, верхней допустимой границей служит H_{max} из Major_Loop.csv. Для убывающего направления используется H_{min} . Тем самым программа не экстраполирует экспериментальные магнитные зависимости за пределы области, определённой исходными данными. Поиск интервала

осуществляется в два этапа. На первом этапе от H_{last} выполняется последовательное продвижение с начальным шагом $initial_H_step$. Если смена знака невязки не обнаружена, величина следующего приращения умножается на коэффициент $bracket_growth_factor$. В текущем варианте используется: $bracket_growth_factor = 1.6$. При этом знак невязки сравнивается между каждыми двумя соседними пробными точками: $F(H_{k-1})F(H_k) \leq 0$. Это существенно аккуратнее простого сравнения всех последующих точек только с H_{last} , поскольку позволяет обнаружить локальную смену знака даже при более сложной форме зависимости $F(H)$.

Если расширяющийся поиск дошёл до границы допустимого диапазона H , но смена знака не обнаружена, выполняется второй этап. Весь интервал между H_{last} и соответствующей границей `Major_Loop.csv` разбивается на `bracket_scan_points` одинаковых частей и проверяется последовательно с более мелким шагом. В текущей версии: $bracket_scan_points = 200$. Такое дополнительное сканирование уменьшает вероятность пропустить узкий интервал смены знака невязки, например вблизи точки разворота или на участке с быстро изменяющейся дифференциальной проницаемостью.

Физически предпочтительное направление первоначального поиска определяется величиной: $drive = V_{next} - RI_{last}$. Если $drive > 0$, сначала рассматриваются $H > H_{last}$; если $drive < 0$, поиск начинается в сторону уменьшения H . Однако это условие используется только для выбора первого направления поиска. Если `bracket` там найти не удаётся, полностью повторяется поиск в противоположном направлении. Если интервал со сменой знака не найден ни в одном направлении, программа не продолжает вычисления с произвольным приближением. Вместо этого рассчитываются значения невязки на обеих границах допустимого диапазона и выдаётся диагностическое сообщение с H_{last} , B_{last} , I_{last} , V_{next} , $drive$, $F(H_{min})$, $F(H_{last})$ и $F(H_{max})$. Это позволяет отличить численную проблему поиска от физической ситуации, когда требуемое решение действительно находится за пределами диапазона, заданного `Major_Loop.csv`.

Решение нелинейного уравнения

После нахождения интервала $[H_a, H_b]$ начинается стандартный метод бисекции. На каждой итерации вычисляется: $H_m = \frac{H_a + H_b}{2}$. Для H_m вновь вызывается `residual()`, то есть заново выполняется полный пробный расчёт модели Tellinen–Madelung, вычисляются B_m , I_m , соответствующее пробное состояние магнитной памяти и $F(H_m)$. Если $F(H_a)F(H_m) \leq 0$, принимается новая правая граница $H_b = H_m$. В противном случае устанавливается $H_a = H_m$. Важнейшим моментом остаётся то, что магнитное состояние от одной итерации бисекции не передаётся следующей итерации. Каждое H_m рассчитывается из одного и того же подтверждённого состояния в момент t_n . Расчёт завершается, если выполняется хотя бы одно из условий: $|F(H_m)| \leq residual_tol$ или $|H_b - H_a| \leq H_tol$. Одновременно во время поиска запоминается точка с минимальным абсолютным значением невязки. Если за `max_bisection_iter` необходимая точность не достигнута, временной шаг считается нерешённым и программа завершается сообщением об ошибке, а не принимает последнее приближение автоматически.

Фиксация подтверждённого состояния

После успешного завершения `solve_time_step()` возвращаются: H_{next} , B_{next} , I_{next} , а также соответствующие именно этому корню `root_next`, `stack_next` и направление движения. Только в этот момент:

```
root_branch = root_next
```

и

```
reversal_stack = stack_next
```

Таким образом, все промежуточные состояния, возникшие при поиске `bracket` и при бисекции, отбрасываются. Если новое поле действительно отличается от предыдущего более чем на H_{tol} , обновляется также `previous_direction`. После этого новые H , B и I становятся подтверждёнными значениями и используются на следующем временном шаге. Именно это разделение подтверждённого и пробного состояния является принципиально важным для совместной работы нелинейного решателя и модели `Madelung`. Последовательность пробных H , создаваемая численным методом, не является реальной временной траекторией магнитного поля и поэтому не должна образовывать ложные точки разворота или стирать существующую магнитную память.

Parameters.py и Parameters.txt

Файл `Parameters.py` выполняет ту же служебную функцию, что и в проекте непосредственного расчёта гистерезисной петли: считывает `Parameters.txt`, различает числовые и строковые параметры и проверяет их допустимость. Строковыми остаются `root_branch` и `IntegrationMethod`. Остальные параметры преобразуются в числовой формат. Для проекта индуктора дополнительно проверяется наличие параметров электрической цепи, временного решателя и алгоритма поиска корня.

Кроме уже описанных параметров `Tellinen–Madelung`, `Parameters.txt` содержит параметры магнитной цепи:

```
N, R, Lm, A, Lg
```

параметры внешнего напряжения:

```
V_offset, V_amplitude, V_frequency
```

параметры временного расчёта:

```
simulation_time, dt
```

а также параметры нелинейного решателя:

```
residual_tol, max_bracket_iter, max_bisection_iter, initial_H_step,  
bracket_growth_factor, bracket_scan_points
```

Параметр `bracket_growth_factor` должен быть больше единицы, а `bracket_scan_points` — не меньше двух; эти условия отдельно контролируются при чтении файла.

V.py

Вместо использовавшегося ранее файла Hscan.py в проекте индуктора появляется файл V.py. Он задаёт уже не историю магнитного поля, а внешнее напряжение на обмотке. В текущей реализации используется функция: $V(t) = V_{\text{offset}} + V_{\text{amplitude}} \sin(2\pi V_{\text{frequency}} t)$. Все параметры источника находятся в Parameters.txt, поэтому форму возбуждения можно изменять независимо от основного алгоритма расчёта.

Выходные файлы

По окончании расчёта main.py создаёт три выходных файла. Current.csv содержит две колонки:

```
time_s, I_A
```

и представляет рассчитанный переходный ток в обмотке. BNloop.csv содержит:

```
H_A_per_m, B_T
```

и сохраняет фактическую магнитную траекторию сердечника, возникшую уже не вследствие заранее заданного $H(t)$, а как результат решения полной электромагнитной задачи.

Дополнительно формируется диагностический файл State.csv:

```
time_s, root_branch, stack_depth
```

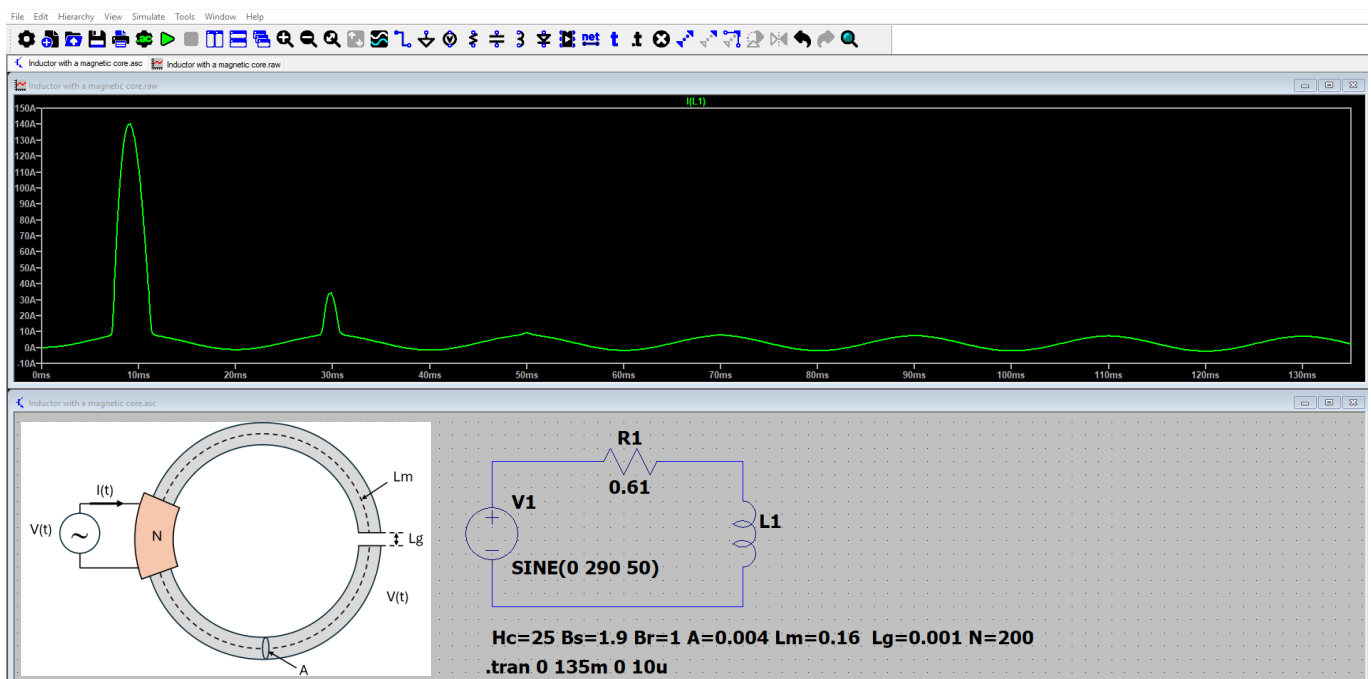
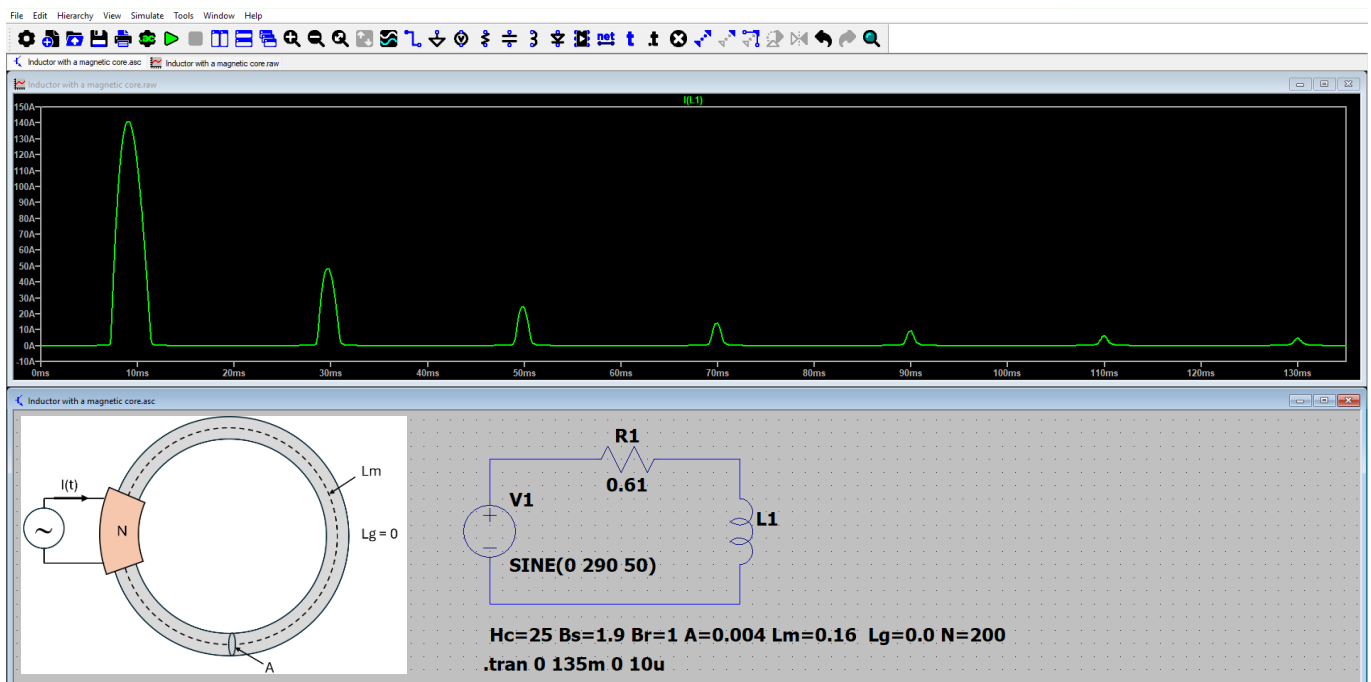
Он позволяет проследить изменение типа корневой ветви и глубины стека Madelung во времени. При этом сам список reversal points в файл не записывается: stack_depth показывает только текущее число активных точек магнитной памяти.

Одновременно программа строит два графика: переходный ток $I(t)$ и полную траекторию $B(H)$. В результате структура программы сохраняет то же разделение функций, которое использовалось в проекте петли гистерезиса. Major_Loop.csv задаёт свойства магнитного материала, BN.py отвечает только за закон Tellinen–Madelung и магнитную память, V.py задаёт внешнее возбуждение, Parameters.txt содержит параметры магнитной цепи и численного расчёта, а main.py решает нелинейное уравнение индуктора и определяет физически принятую последовательность магнитных состояний. Такое разделение позволяет изменять модель материала, электрические параметры или закон возбуждения независимо друг от друга, не перестраивая основной алгоритм.

Сравнение расчетов с симуляциями индуктора в LTspice

Мы протестировали модель Tellinen–Madelung для схемы индуктора, которая использовалась в патенте [US 7502723B1](#) [6] для демонстрации преимущества модели Chan-Engelhardt [3] по сравнению с оригинальным Chan [2]. Модель Chan-Engelhardt реализована в LTspice. Симуляции индуктора в LTspice показаны ниже, где рассчитывался переходный ток через индуктор с небольшим сопротивлением катушки 0.61Ω и числом витков $N = 200$ в момент включения синусоидального генератора. Последующие за этим стабильные синусоидальные осцилляции лежат за пределами рассматриваемого временного интервала и не показаны. Первый график для магнитного сердечника без щели, а второй с щелью. Геометрические и

магнитные параметры сердечника показаны на рисунках схемы: A – поперечное сечение сердечника (m^2), L_m – длина сердечника по средней линии (m), L_g – длина щели (m), H_c – коэрцитивность (A/m), B_r – остаточная намагниченность (T), B_s – намагниченность насыщения (T). График – зависимость тока (A) через индуктор от времени (s). Генератор синусоидального напряжения имел амплитуду 290 V , частоту 50 Hz , и нулевую фазу, т.е. стартовал с нуля в момент времени $t = 0$. Таким образом, симуляция намагниченности начиналась из полностью размагниченного состояния. График $I(t)$ – зависимость тока (A) через индуктор от времени (s). Файлы LTspice сохранены в проектной папке «LTspice_Inductor with a magnetic core» в репозитории [GitHub](#).^[4]



Наши расчеты полностью совпали с симуляциями в LTspice, как показано на рисунках ниже.
Используемые в алгоритме Python параметры:

```
# =====
# INITIAL MAGNETIC STATE
# =====

# root_branch:
#   initial
#   upper_major
#   lower_major
root_branch=initial

# Initial magnetic field strength [A/m]
Hstart=0.0

# =====
# MAGNETIC CORE AND COIL PARAMETERS
# =====

# Number of coil turns [-]
N=200

# Coil resistance [ohm]
R=0.61

# Mean magnetic path length in ferromagnetic core [m]
Lm=0.16

# Effective magnetic cross-sectional area [m^2]
A=0.004

# Air-gap length [m]
Lg=0.0

# =====
# VOLTAGE SOURCE PARAMETERS
# =====

# DC voltage offset [V]
V_offset=0.0

# Sinusoidal voltage amplitude [V]
V_amplitude=290.0

# Voltage frequency [Hz]
V_frequency=50.0

# =====
# SIMULATION PARAMETERS
# =====

# Total simulation time [s]
simulation_time=0.135

# Time step [s]
# 1e-5 gives 2000 steps per 50-Hz period.
# Reduce it for a time-step convergence study.
dt=0.00001

# =====
# TELLINEN-MADELUNG NUMERICAL PARAMETERS
# =====

# H comparison tolerance [A/m]
H_tol=1.0e-10

# B comparison / envelope tolerance [T]
B_tol=1.0e-9

# Minimum admissible major-loop width [T]
D_tol=1.0e-12
```

```

# Numerical Stieltjes integral method:
#   trapezoid
#   lagrange
IntegrationMethod=lagrange

# Number of integration intervals per Tellinen integral
IntegrationIntervals=200

# =====
# NONLINEAR SOLVER PARAMETERS
# =====

# Electrical residual tolerance [V]
residual_tol=1.0e-9

# Maximum number of root-bracketing iterations
max_bracket_iter=80

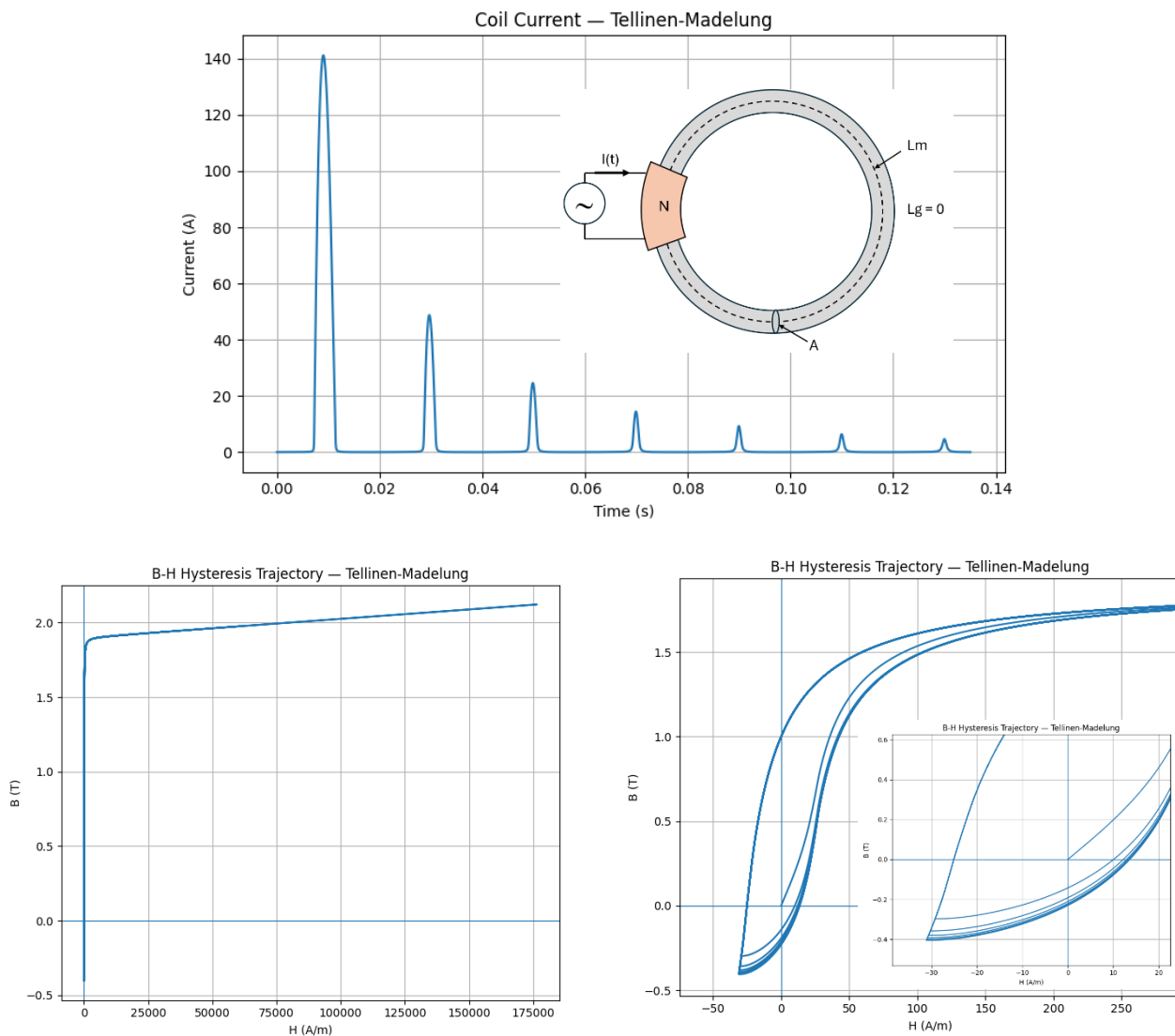
# Maximum number of bisection iterations
max_bisection_iter=120

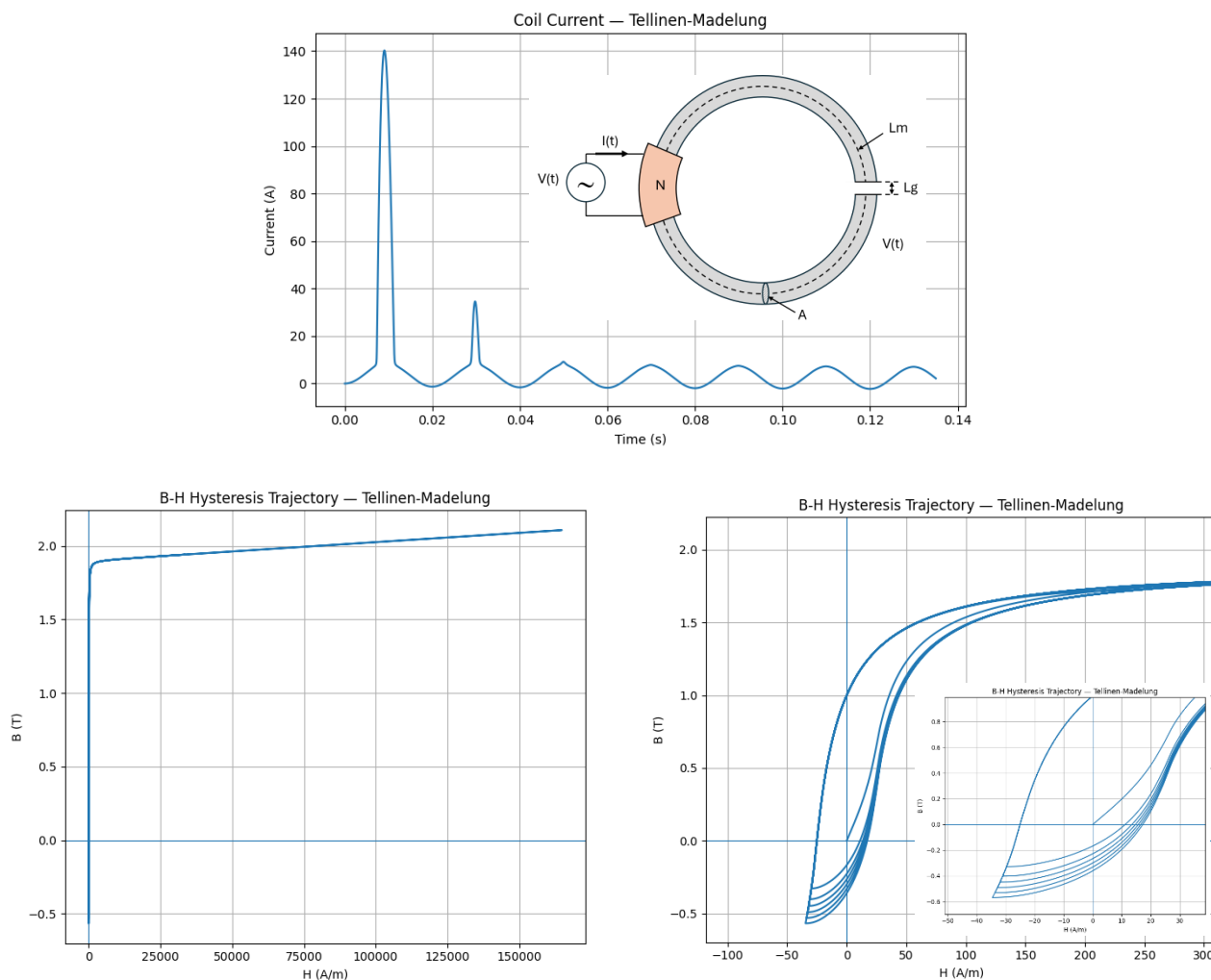
# Initial H increment for root bracketing [A/m]
initial_H_step=6.0

# Multiplicative growth factor for the outward bracket search
bracket_growth_factor=1.6

# Number of intervals in the fallback directional H scan
bracket_scan_points=200

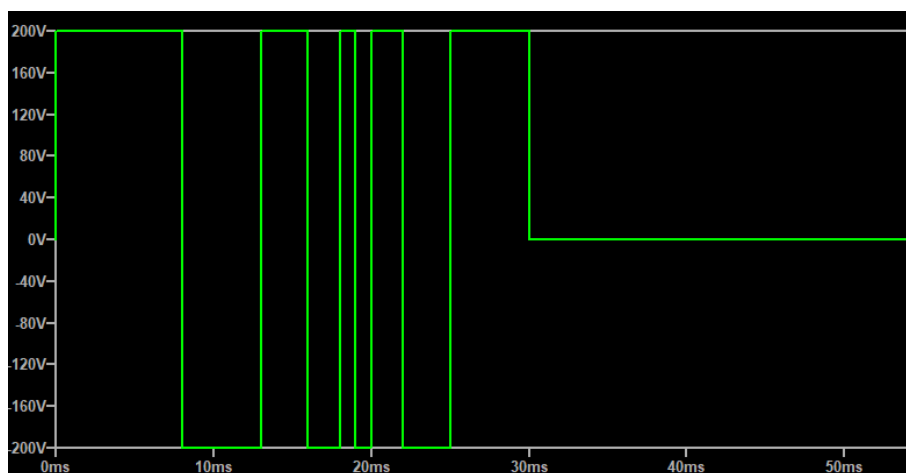
```





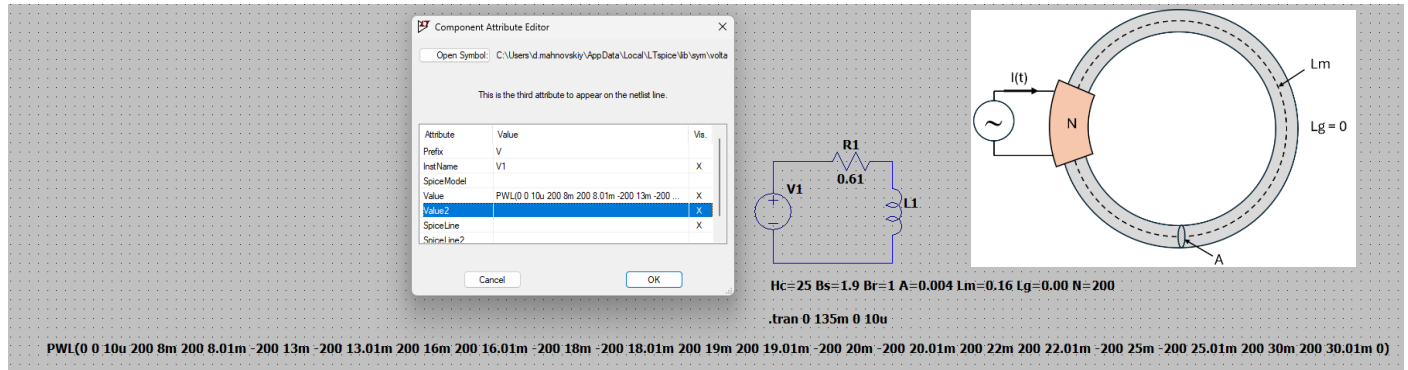
В LTspice используется модель Chan-Engelhardt с локальной магнитной памятью (не запоминает точки разворота). Однако мы воспроизвели те же самые зависимости $I(t)$, используя модель Tellinen-Madelung с глубокой магнитной памятью. Причем, даже различающие наклоны минорных петель в моделях не отразились на профиле тока. Для выявления возможного эффекта глубокой магнитной памяти на электрические переходные процессы можно попытаться использовать более экзотическое возбуждение.

Рассмотрим сложное PWM возбуждение $V(t)$, показанное на рисунке ниже. Оно будет формировать вложенные минорные петли в обоих моделях, но по различным сценариям.



В LTspice эта последовательность прямоугольных импульсов задается следующей командной строкой в атрибутах источника Value:

```
PWL(0 0 10u 200 8m 200 8.01m -200 13m -200 13.01m 200 16m 200 16.01m -200 18m -200 18.01m 200 19m 200 19.01m -200 20m -200 20.01m 200 22m 200 22.01m -200 25m -200 25.01m 200 30m 200 30.01m 0)
```



В Python-модуле V.py это возбуждение может быть записано в следующем виде:

```
def V(t):

    V0 = par["V_test"]

    if t < 0.008:
        return +V0

    elif t < 0.013:
        return -V0

    elif t < 0.016:
        return +V0

    elif t < 0.018:
        return -V0

    elif t < 0.019:
        return +V0

    elif t < 0.020:
        return -V0

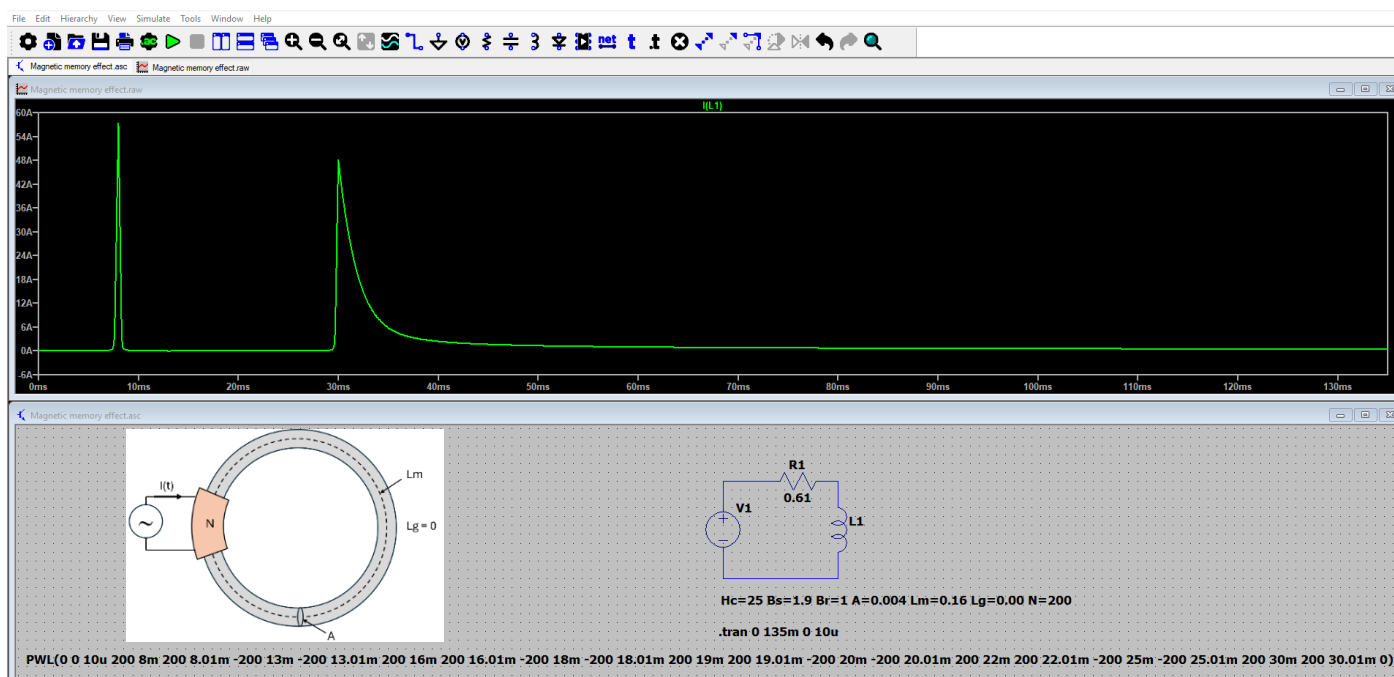
    elif t < 0.022:
        return +V0

    elif t < 0.025:
        return -V0

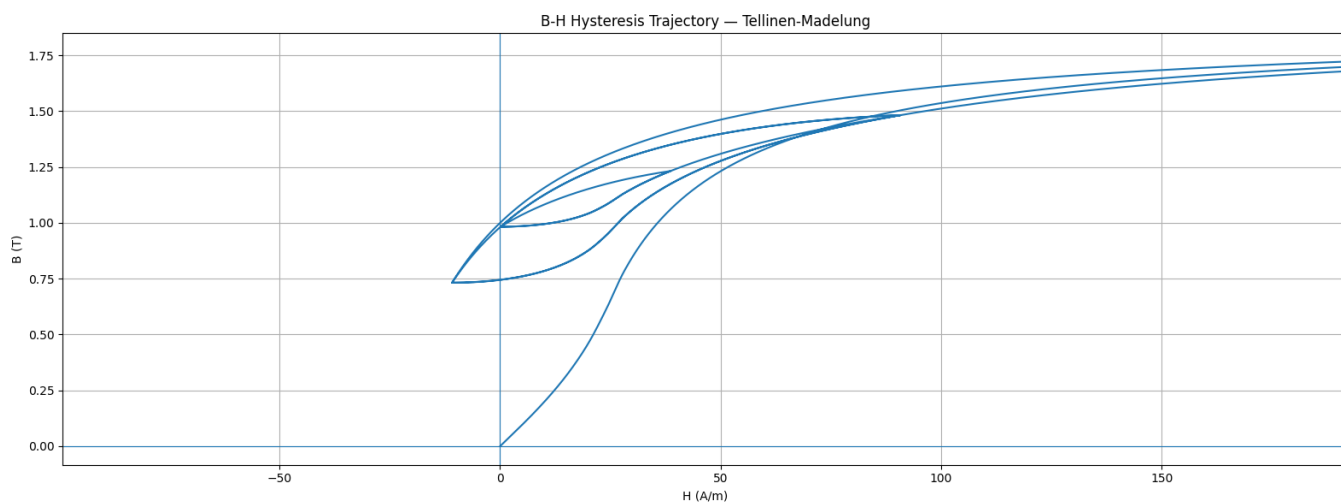
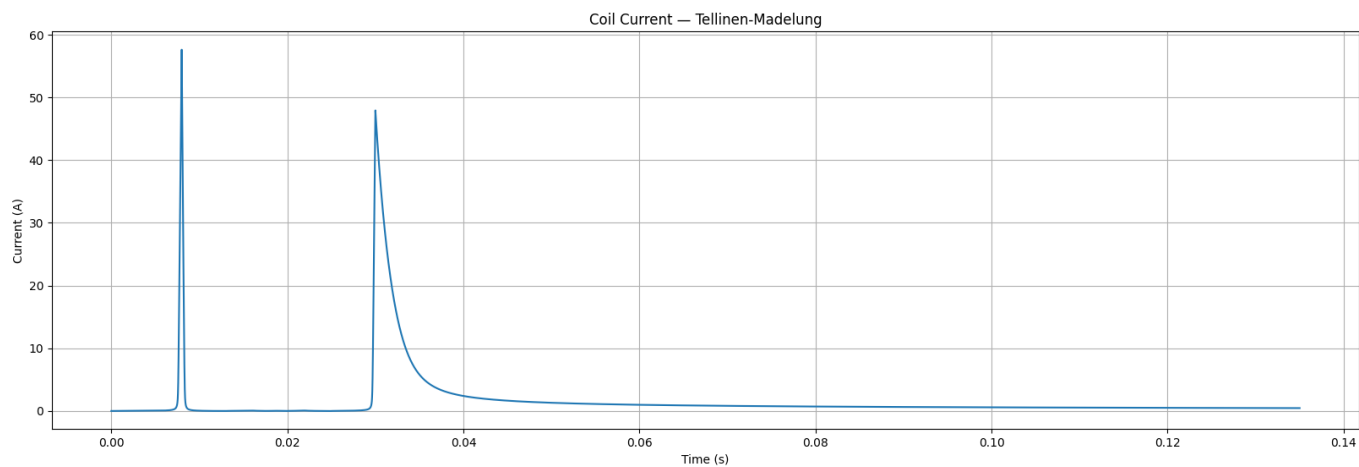
    elif t < 0.030:
        return +V0

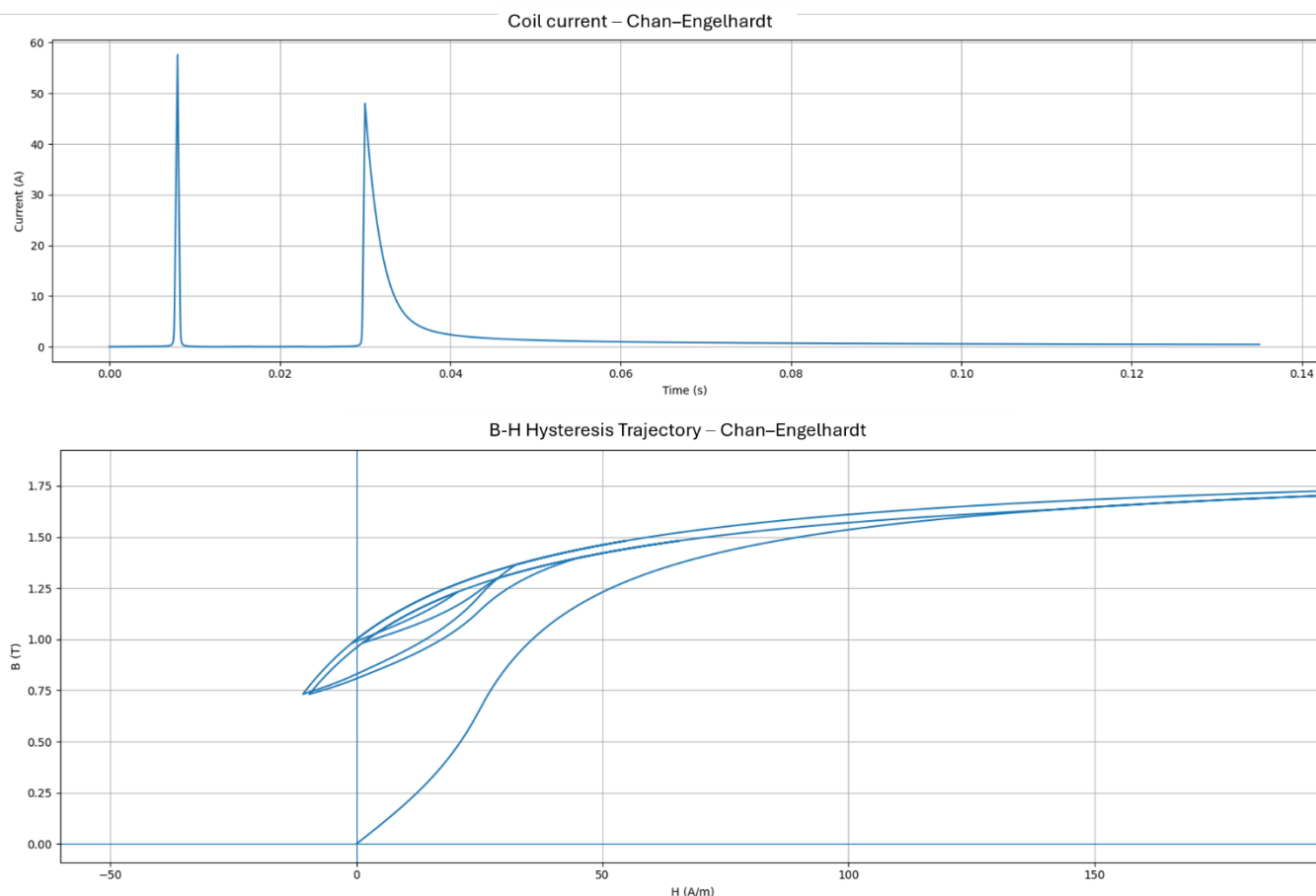
    else:
        return 0.0
```


Профиль тока $I(t)$ через индуктор, вычисленный в LTspice по модели Chan-Engelhardt, показан ниже.



Теперь сделаем эти же симуляции нашими Python алгоритмами для обеих моделей, где в качестве мажорной петли будет использоваться промоделированная петля Chan.





Отклики тока для обеих моделей совпадают. Таким образом, проведённые расчёты не выявили заметного влияния различий в реализации магнитной памяти на переходный ток индуктора. Несмотря на различие внутренних алгоритмов хранения истории намагничивания, модели Tellinen–Madelung и Chan–Engelhardt при рассмотренных последовательностях возбуждения дают совпадающие электрические отклики.

Проведённое сравнение показывает, что в задачах схемотехнического моделирования влияние детальной реализации магнитной памяти на электрический отклик может быть значительно менее существенным, чем корректное описание основных нелинейных свойств магнитной цепи — насыщения, коэрцитивности, остаточной индукции и правдоподобного поведения минорных петель. Поэтому для многих задач электроники, по-видимому, нет необходимости стремиться к максимально сложным и формально совершенным моделям магнитной памяти, если более простая модель обеспечивает адекватное воспроизведение наблюдаемых токов и напряжений. Для расчётов магнитного встряхивателя это обстоятельство особенно важно. Оно означает, что результат может оказаться достаточно устойчивым к ряду упрощающих предположений при моделировании магнитной цепи, включая использование приближённых моделей гистерезиса и ограниченного набора параметров материала. В таком случае основное внимание целесообразно сосредоточить на тех факторах, которые непосредственно определяют силовой и переходный отклик актуатора: геометрии магнитной цепи, величине и изменении воздушных зазоров, насыщении отдельных участков сердечника, сопротивлении и параметрах обмотки, а также на корректном учёте внешней электрической цепи.

Источники

- [1] G. Mörée and M. Leijon, “*Review of Hysteresis Models for Magnetic Materials*”, *Energies* 2023, 16, 3908; <https://www.mdpi.com/1996-1073/16/9/3908>
- [2] John H. Chan et al., «*Nonlinear Transformer Model for Circuit Simulation*», *IEEE Transactions on Computer-Aided Design*, vol. 10, No. 4, APRIL 1991.
- [3] Репозиторий модели Chan-Engelhardt: <https://github.com/DmitriyMakhnovskiy/Chan-Engelhardt-Model-BHloops>
- [4] Репозиторий модели Tellinen–Madelung: <https://github.com/DmitriyMakhnovskiy/Tellinen-Madelung-Model-BHloops>
- [5] K. Kelp et al., “*Modeling Pulsed Magnetic Core Behavior in LTspice*”, *Electronics* 2025, 14, 2335; <https://www.mdpi.com/2079-9292/14/12/2335>
- [6] M. Th. Engelhardt, “*Asymmetric Minor Hysteresis Loop Model and Circuit Simulator Including the Same*”, U.S. Patent No. 7502723B1, Mar. 10, 2009; <https://patents.google.com/patent/US7502723B1/en>